

252-0027

Einführung in die Programmierung
Übungen

Woche 10: Verlinkte Objekte, Klassen

Joran Bühler
Departement Informatik
ETH Zürich

Meine Bonus Implementation

LinkedLists

LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

LinkedLists

Referenz auf Objekt des selben Typs!

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

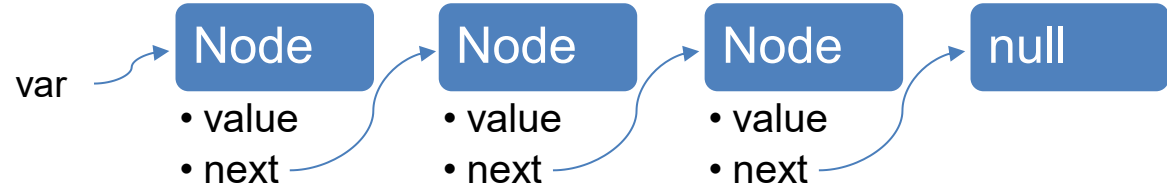
single linked list Knotenaufbau

LinkedLists

Referenz auf Objekt des selben Typs!

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau



LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

Beispiel

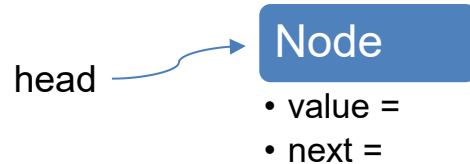
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

Beispiel



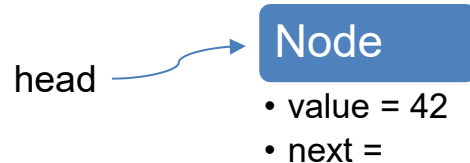
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

Beispiel



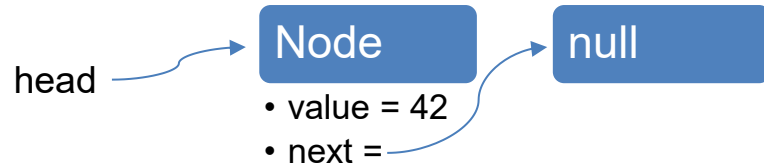
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

Beispiel



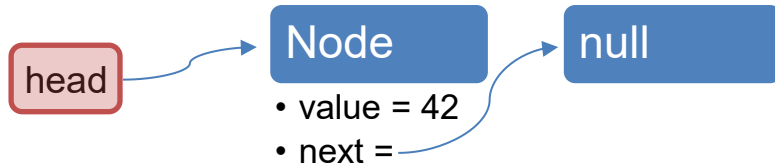
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

Beispiel



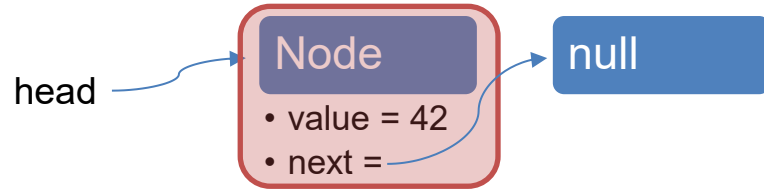
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

Beispiel



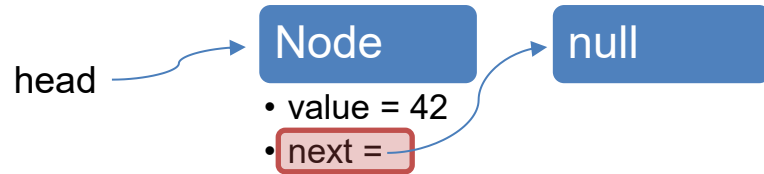
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

Beispiel



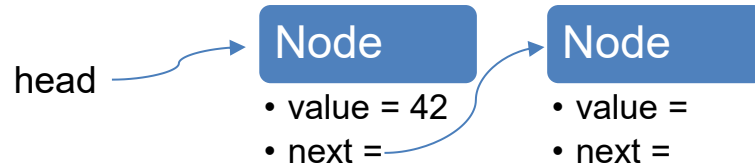
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

Beispiel



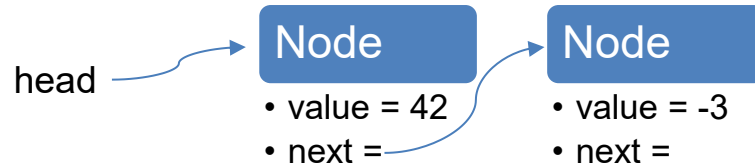
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

Beispiel



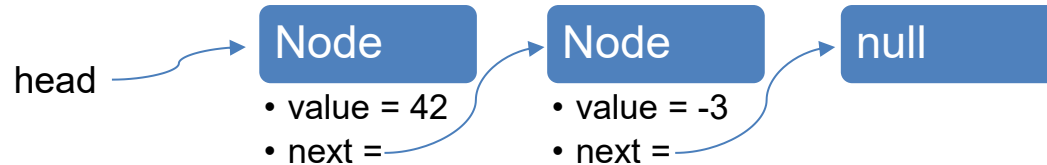
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

Beispiel



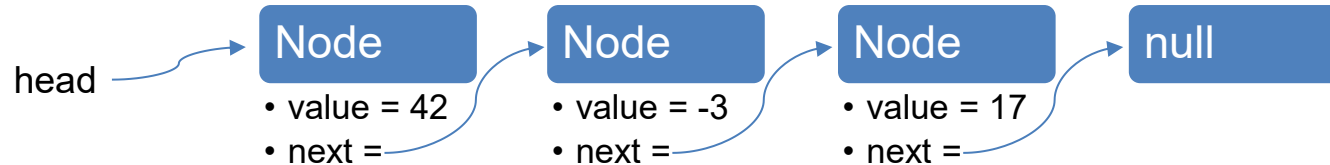
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

Beispiel

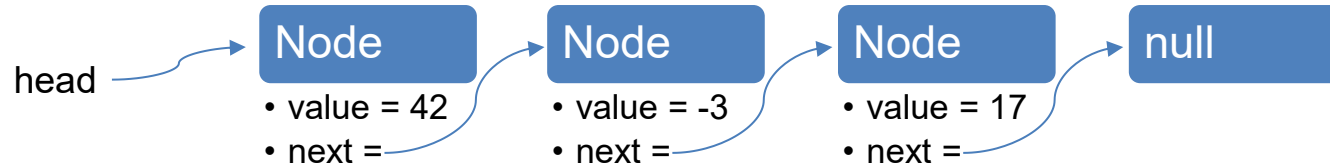


LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

head.next.next = new Node(15);
was passiert nun?

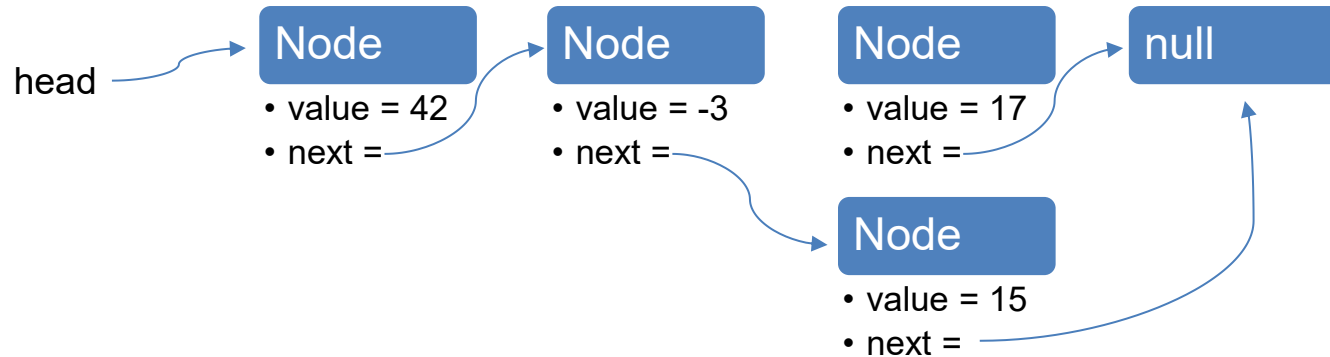


LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
head.next.next = new Node(15);  
was passiert nun?
```



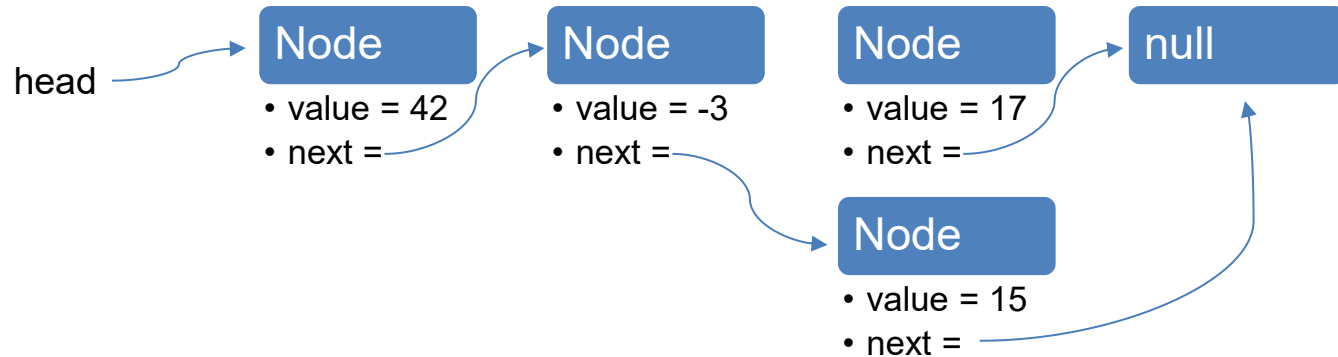
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

`head.next.next = new Node(15);`
was passiert nun?

`head.next = head.next.next;`
was passiert nun?



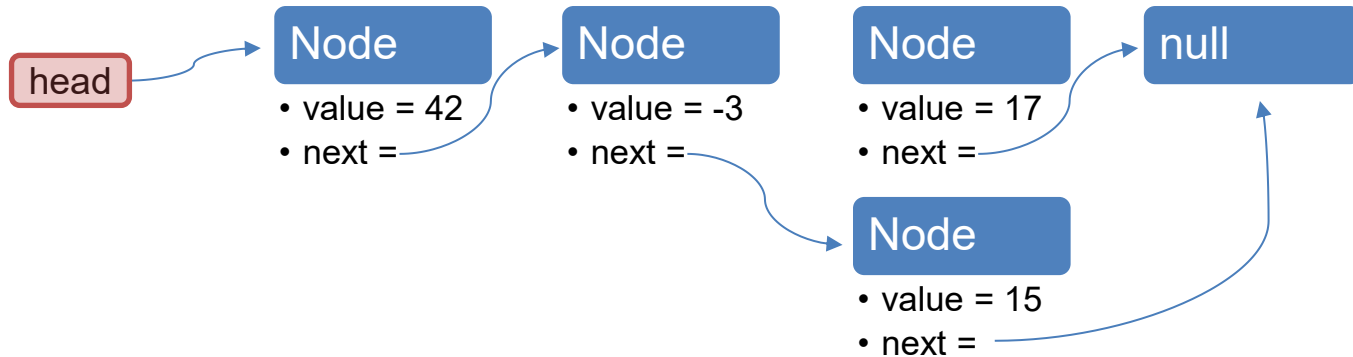
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

`head.next.next = new Node(15);`
was passiert nun?

`head.next = head.next.next;`
was passiert nun?



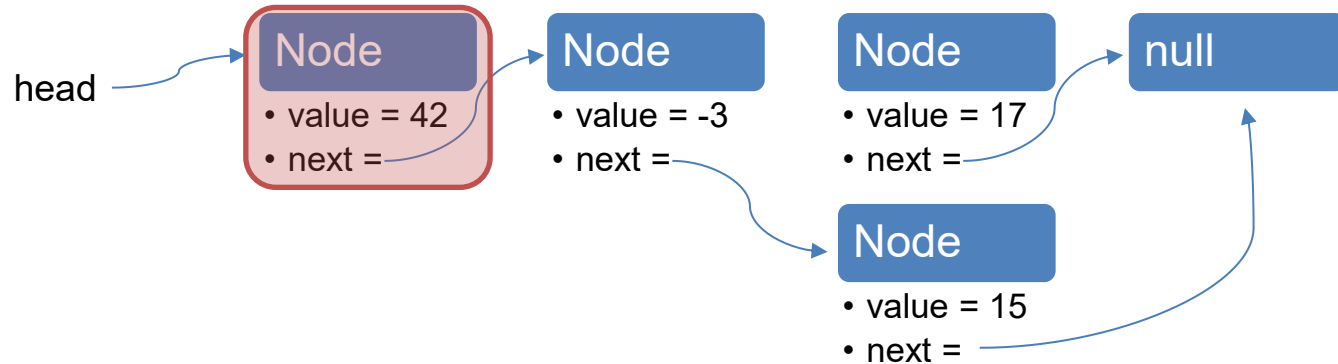
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

```
head.next.next = new Node(15);  
was passiert nun?
```

```
head.next = head.next.next;  
was passiert nun?
```



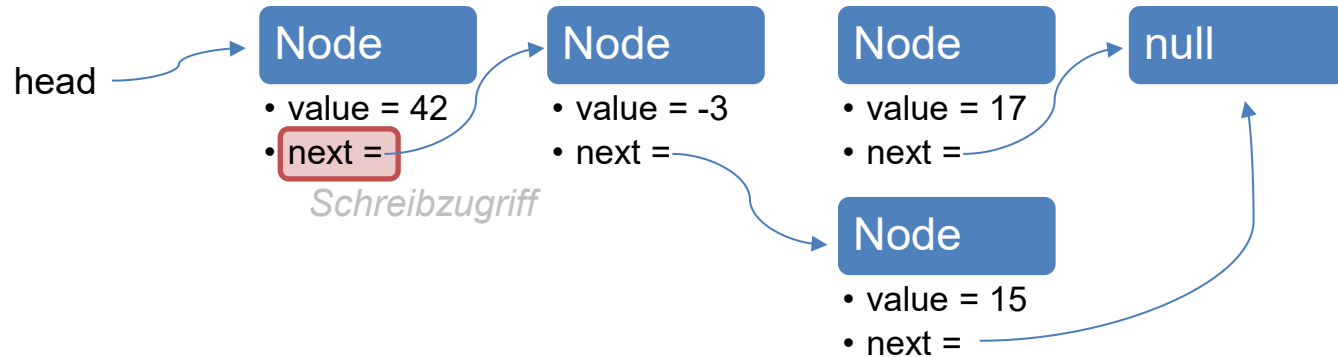
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

head.next.next = new Node(15);
was passiert nun?

head.next = head.next.next;
was passiert nun?



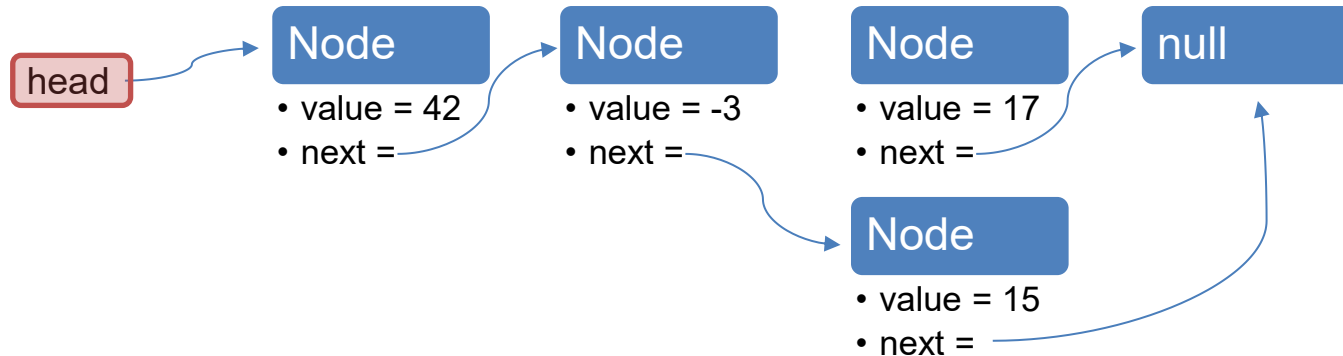
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

`head.next.next = new Node(15);`
was passiert nun?

`head.next = head.next.next;`
was passiert nun?



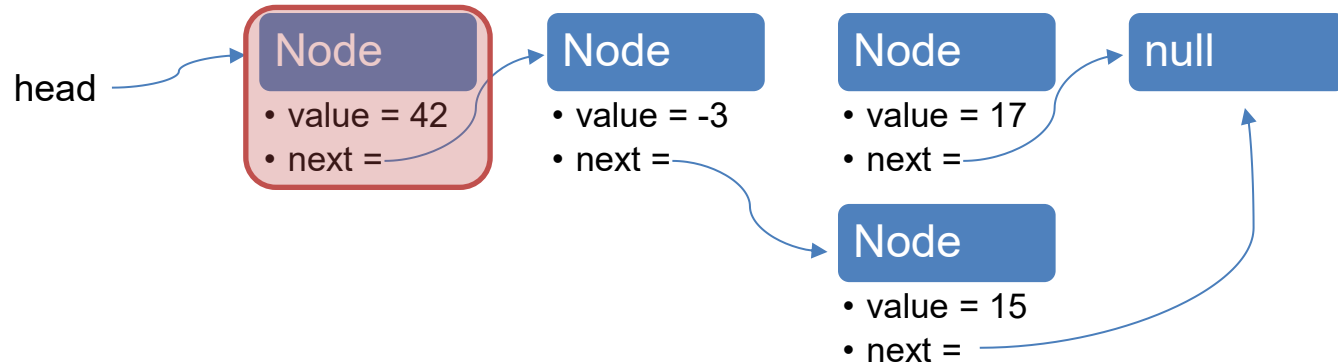
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

`head.next.next = new Node(15);`
was passiert nun?

`head.next = head.next.next;`
was passiert nun?



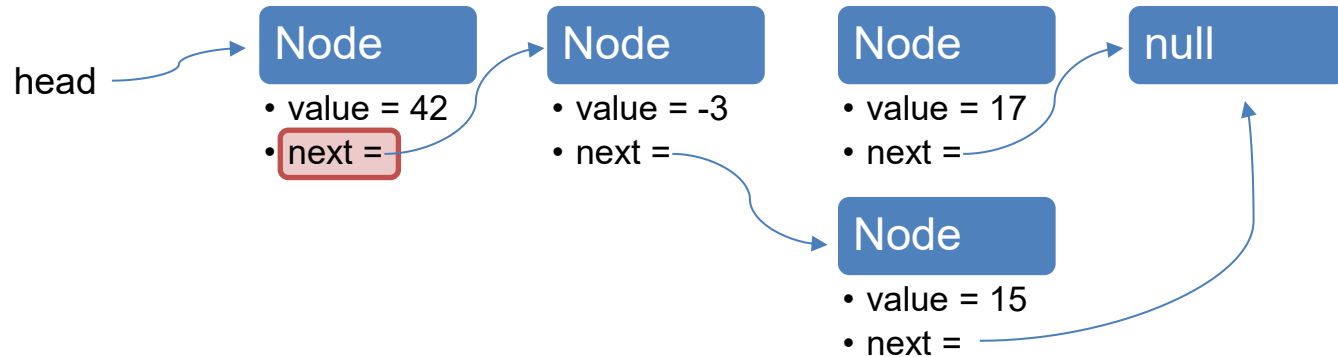
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

`head.next.next = new Node(15);`
was passiert nun?

`head.next = head.next.next;`
was passiert nun?



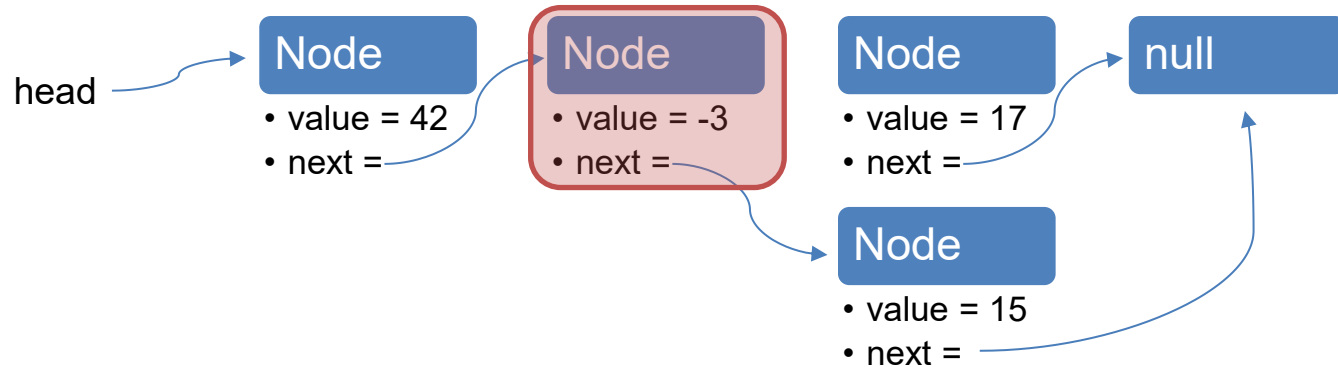
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

`head.next.next = new Node(15);`
was passiert nun?

`head.next = head.next.next;`
was passiert nun?



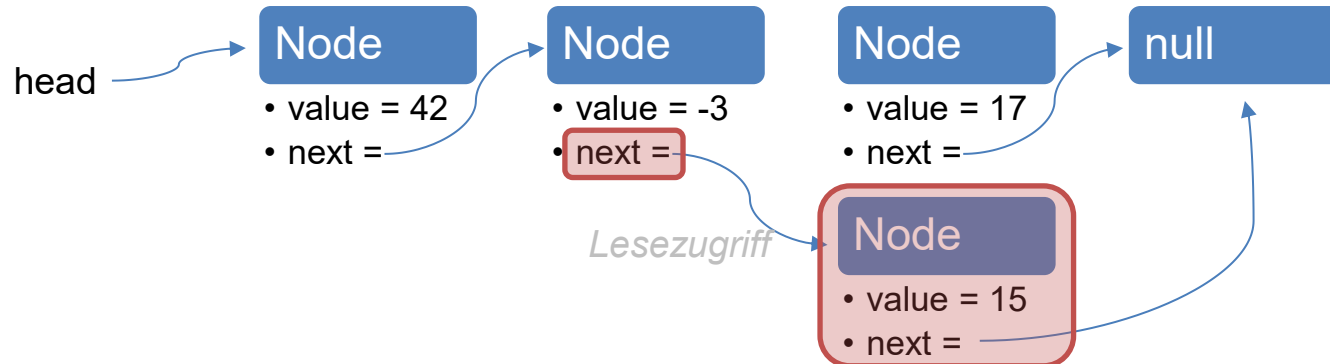
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

`head.next.next = new Node(15);`
was passiert nun?

`head.next = head.next.next;`
was passiert nun?



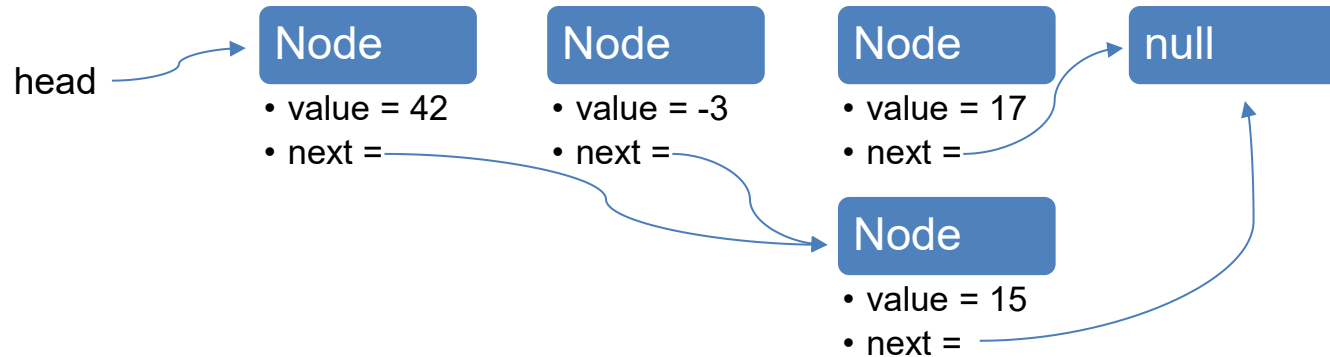
LinkedLists

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

single linked list Knotenaufbau

`head.next.next = new Node(15);`
was passiert nun?

`head.next = head.next.next;`
was passiert nun?

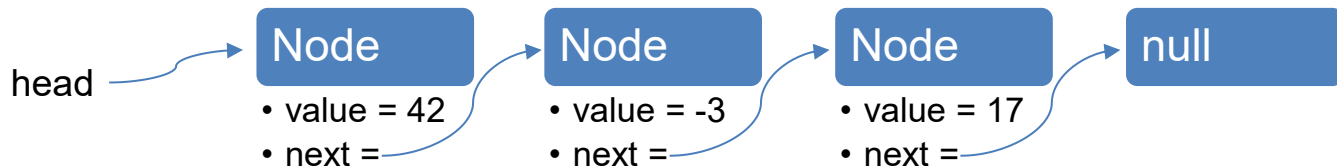


LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



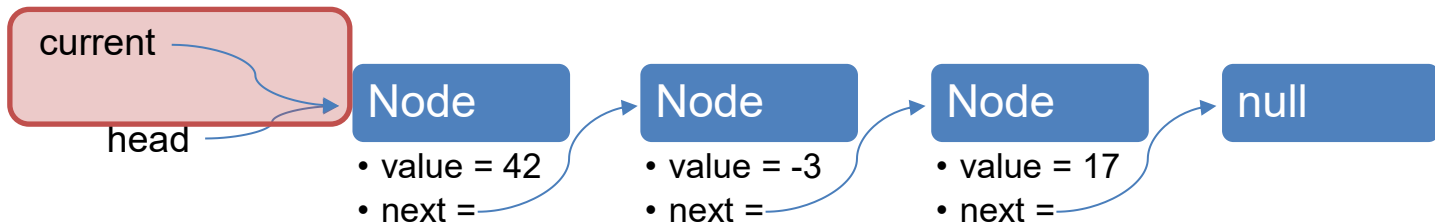
Output:

LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



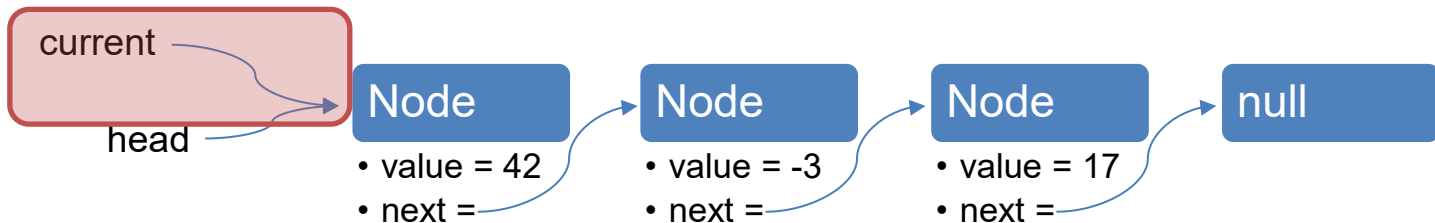
Output:

LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



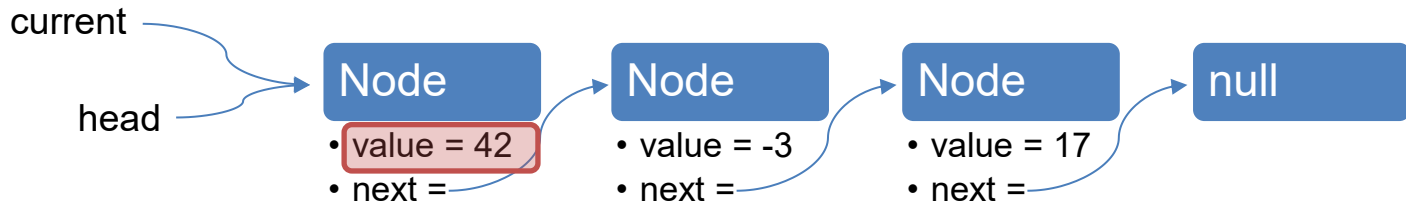
Output:

LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



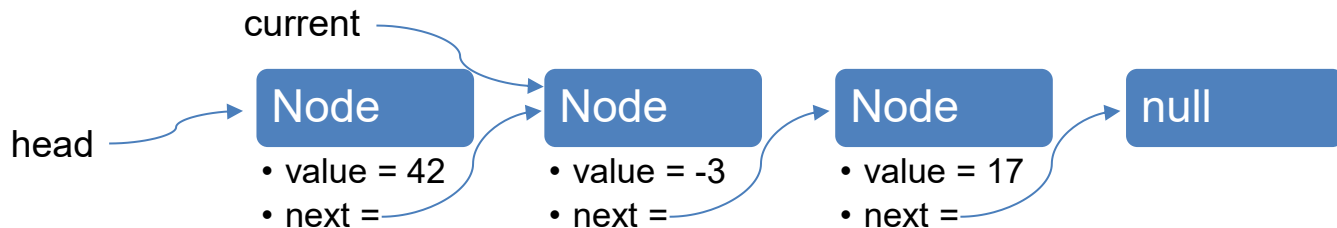
Output: 42

LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



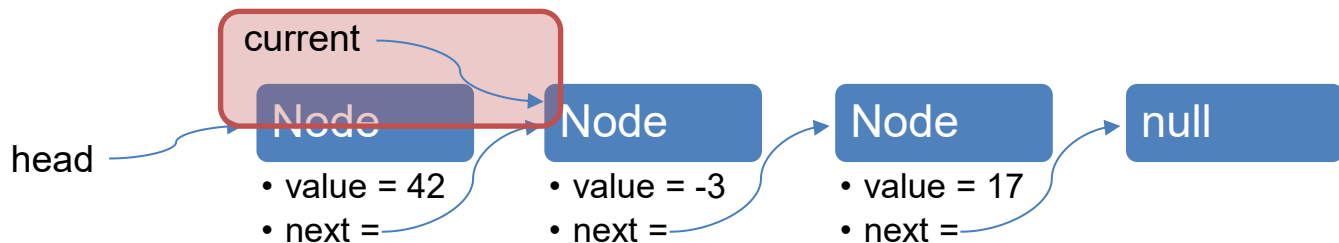
Output: 42

LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



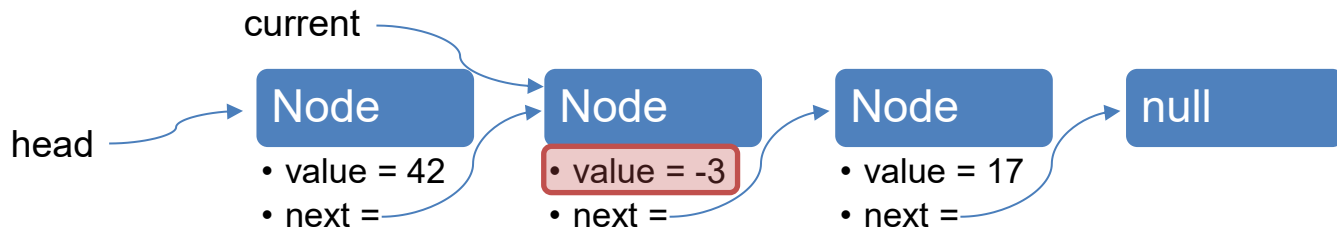
Output: 42

LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



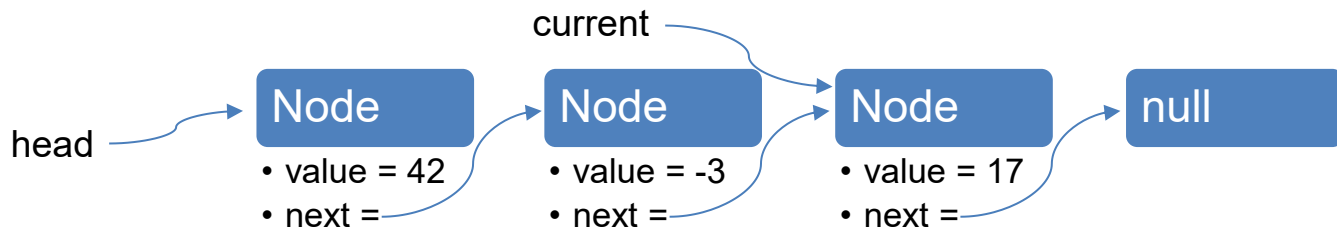
Output: 42 -3

LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



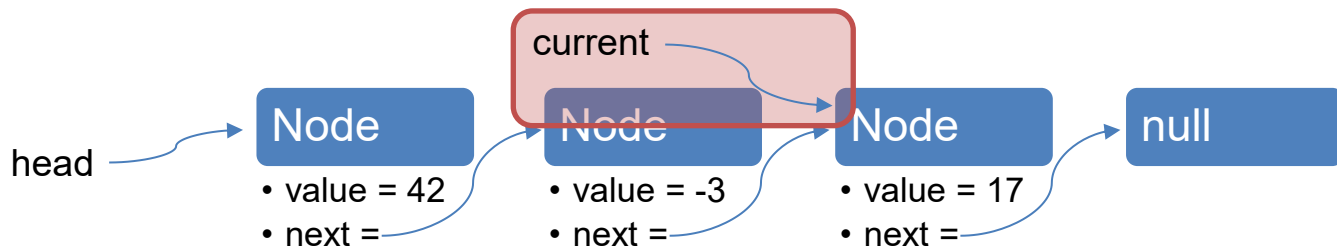
Output: 42 -3

LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



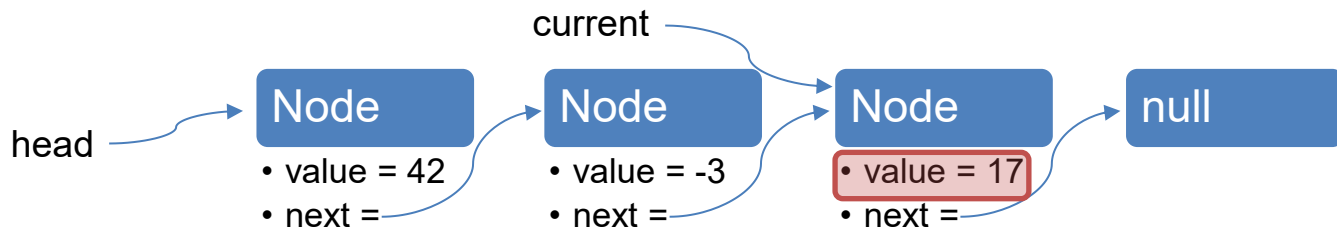
Output: 42 -3

LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



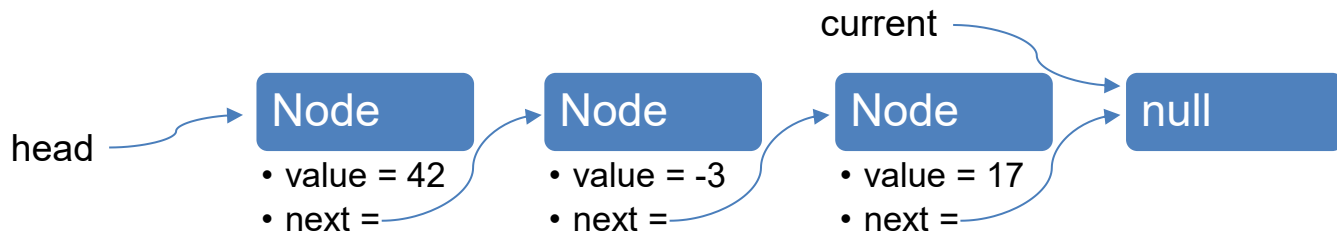
Output: 42 -3 17

LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



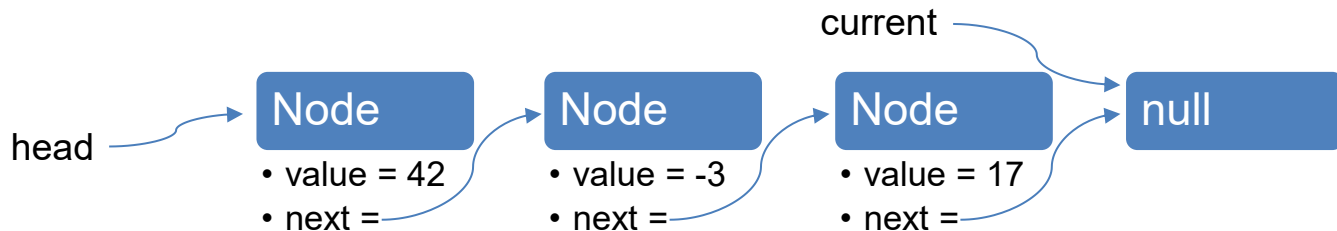
Output: 42 -3 17

LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



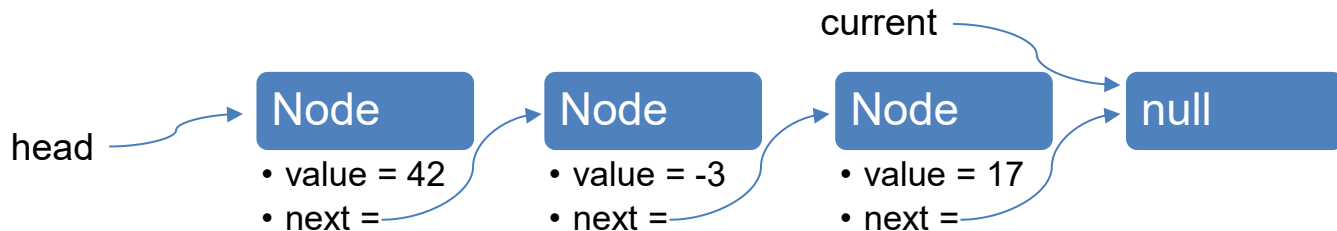
Output: 42 -3 17

LinkedLists durchlaufen

```
class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

```
Node head = new Node(42);  
head.next = new Node(-3);  
head.next.next = new Node(17);
```

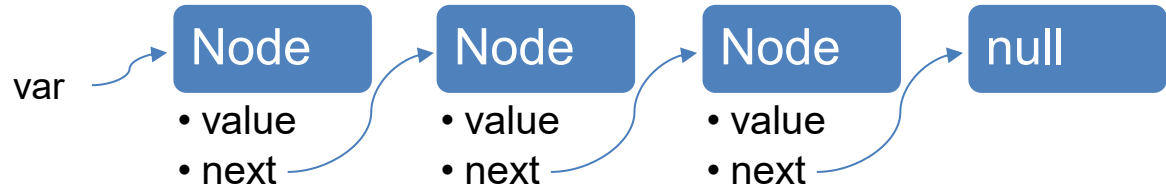
```
Node current = head;  
while (current != null) {  
    System.out.println(current.value);  
    current = current.next;  
}
```



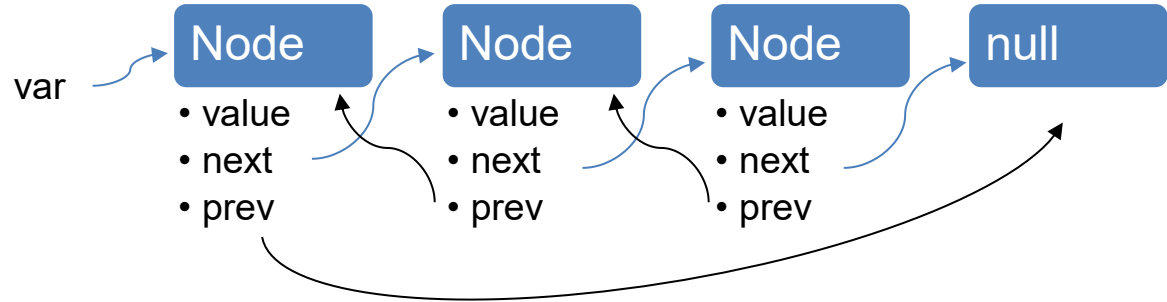
Output: 42 -3 17

Verschiede Typen von LinkedLists

single LinkedList



double LinkedList



LinkedLists – bereits implementiert

- Implementation ist eine double linked list
- Import mit

```
import java.util.LinkedList;
```

- Initialisierung mit

```
LinkedList<Type> name = new LinkedList<Type>();
```



„Type“ muss ein **Referenztyp** sein

LinkedLists – bereits implementiert

- Implementation ist eine double linked list

- Import mit

```
import java.util.LinkedList;
```

- Initialisierung mit

```
LinkedList<Type> name = new LinkedList<Type>();
```

Diesen Typ
kann man auch
weglassen



„Type“ muss ein **Referenztyp** sein

LinkedLists – Methoden

Methode (LinkedList<String>)	Bedeutung
<pre>list.addFirst("C"); list.add(1, "B"); list.add("A");</pre>	Hinzufügen eines Elements - am Anfang - an einer bestimmten Position - am Ende
<pre>list.removeFirst(); list.remove(1); list.removeLast();</pre>	Entfernen eines Elementes - am Anfang - an einer bestimmten Position - am Ende
<pre>String element = list.get(0);</pre>	Lesen eines Elementes an einer bestimmten Position
<pre>int size = list.size();</pre>	Länge der Liste
<pre>list.contains("A");</pre>	Überprüfen, ob ein Element enthalten ist
<pre>list.clear();</pre>	Liste leeren

alle Methoden siehe <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/LinkedList.html>

ArrayLists

ArrayLists

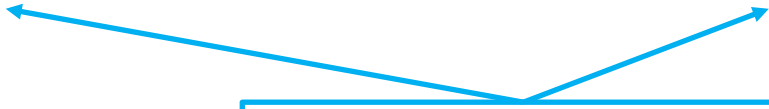
- Implementation via Arrays
- Import mit

```
import java.util.ArrayList;
```

- Initialisierung mit

```
ArrayList<Type> name = new ArrayList<Type>();
```

Diesen Typ
kann man auch
weglassen



„Type“ muss ein **Referenztyp** sein

ArrayLists – Methoden

Die meisten Methoden sind absolut gleich!

alle Methoden siehe <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html>

ArrayLists – Methoden

	Array	ArrayList
Grösse	fixe Grösse	dynamische Grösse
speichert...	primitive Datentypen oder Objekte	nur Objekte (unterstützt Wrapper-Klassen)
Deklaration	<code>int[] myArray = new int[3];</code>	<code>ArrayList<Integer> myArrayList = new ArrayList<>();</code>
Lesen	<code>int x = myArray[0];</code>	<code>myArrayList.get(0);</code>
Schreiben	<code>myArray[0] = 3;</code>	<code>myArrayList.set(0, 3);</code> // 3 at index 0
Länge	<code>myArray.length;</code> // field	<code>myArrayList.size();</code> // method
Element hinzufügen	---	<code>myArrayList.add(35);</code> // 35 at the end <code>myArrayList.add(1, 35);</code> // 35 at index 1
Element entfernen	---	<code>myArrayList.remove(1);</code> // per index or <code>myArrayList.remove(e);</code> // per object e
print	<code>Arrays.toString(myArray);</code>	<code>myArrayList.toString();</code>

alle Methoden siehe <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html>

LinkedList vs ArrayList

LinkedList vs ArrayList

Eigenschaft	LinkedList	ArrayList
Im Speicher	Doppelt verkettete Liste	Array
Zugriffszeit	$O(n)$	$O(1)$
Einfügen an einer beliebigen Position	$O(n)$	$O(n)$ wegen Verschiebung der Elemente
Einfügen am Anfang	$O(1)$	$O(n)$
Löschen am Anfang	$O(1)$	$O(n)$
Einfügen am Ende	$O(1)$	$O(1)$ oder $O(n)$
Löschen am Ende	$O(1)$	$O(1)$

LinkedList vs ArrayList

Für die Implementierung von **Stacks** eignet sich eine _____, da Einfügen und Entfernen am Anfang/Ende $O(1)$ sind.

Für **Queues** ist eine _____ besser geeignet, da sie effizient Einfügen am Ende und Entfernen am Anfang unterstützt, ohne Elemente zu verschieben.

LinkedList vs ArrayList

Für die Implementierung von **Stacks** eignet sich eine **LinkedList**, da Einfügen und Entfernen am Anfang/Ende $O(1)$ sind.

Für **Queues** ist eine _____ besser geeignet, da sie effizient Einfügen am Ende und Entfernen am Anfang unterstützt, ohne Elemente zu verschieben.

LinkedList vs ArrayList

Für die Implementierung von **Stacks** eignet sich eine **LinkedList**, da Einfügen und Entfernen am Anfang/Ende $O(1)$ sind.

Für **Queues** ist eine **LinkedList** besser geeignet, da sie effizient Einfügen am Ende und Entfernen am Anfang unterstützt, ohne Elemente zu verschieben.

Operation	LinkedList list	LinkedList list
Liste als Stack (Element entfernen)	<code>list.pop()</code>	<code>list.removeFirst()</code>
Liste als Stack (Element e hinzufügen)	<code>list.push(e)</code>	<code>list.addFirst(e)</code>
Liste als Queue (Element entfernen)	<code>list.poll()</code>	<code>list.removeFirst()</code>
Liste als Queue (Element e hinzufügen)	<code>list.add(e)</code>	<code>list.add(e)</code>

Prüfungsaufgabe

Gegeben sei das folgende Programmsegment.

```
ArrayList<Integer> al = new ArrayList<>();  
for (int k = 9; k >= 0; k--) {  
    al.add(Integer.valueOf(k+1));  
}  
  
for (int i = 0; i < 2; i++) {  
    for (int k = al.size()-1; k >= 0; k = k-2) {  
        al.remove(Integer.valueOf(k));  
    }  
}  
  
for (Integer k : al) {  
    System.out.print("_" + k); // _____  
}  
System.out.println();
```

Was gibt die letzte Schleife aus? Schreiben Sie die Antwort auf die Zeile oben. Leerzeichen und Zwischenräume sind nicht wichtig, aber beachten Sie bitte, dass Zahlen durch einen Zwischenraum getrennt sind.

Kahoot

- <https://create.kahoot.it/share/21-u9-eprog-mschoeb/46d2df0a-83de-4a10-a5b0-43c58fd202c9>

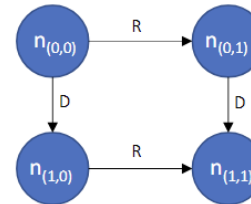
Vorbesprechung

Aufgabe 1: Square Grid

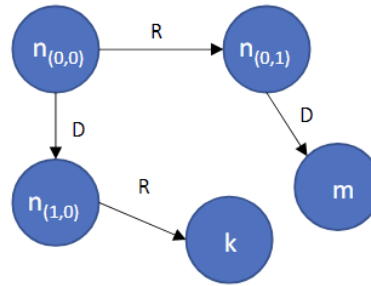
In dieser Aufgabe betrachten wir gerichtete Graphen, wobei es für jeden Knoten g höchstens zwei gerichtete Kanten von g zu anderen Knoten f, h geben kann (f, g, h können gleich sein). Wir unterscheiden dabei zwischen der rechten und der unteren Kante (und damit dem rechten und dem unteren Knoten).

Die Klasse `Node` repräsentiert einen Knoten in einem solchen Graphen. Die Methode `Node.getRight()` (bzw. `Node.getDown()`) gibt den rechten Knoten (bzw. unteren Knoten) zurück (als `Node`-Objekt). Wenn der rechte Knoten von n_0 nicht existiert, dann gibt `Node.getRight()` `null` zurück (analog für den unteren Knoten). Die Methode `Node.setRight(Node r)` (bzw. `Node.setDown(Node d)`) setzt den rechten (bzw. unteren) Knoten.

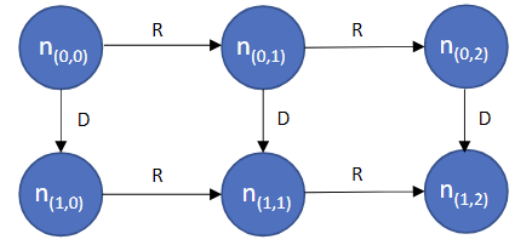
Das Ziel der Aufgabe ist, einen von einem `Node`-Objekt definierten Graphen zu analysieren. Konkret geht es darum, die Grösse des grössten quadratischen Gitters in dem Graphen zu bestimmen, der mit dem übergebenen `Node`-Objekt beschrieben wird, welches den gleichen Ursprungsknoten wie der Graph hat.



Aufgabe 1: Square Grid



(a)



(b)

Abbildung 2: Graphen mit quadratischen Gittern als Teilgraphen

Aufgabe 2: Umkehrung

In einem vorherigen Übungsblatt haben Sie eine Linked List für Integers implementiert. In dieser Aufgabe fügen Sie dieser `LinkedList` eine weitere Methode hinzu, welche die Liste umkehrt. Eine Liste gilt als umgekehrt, wenn für jedes Paar von Nodes `a` und `b`, für welche zuvor `a == b.next` gegolten hat, in der neuen (umgekehrten) Liste `b == a.next` gilt. Zusätzlich entspricht nach der Umkehrung der erste Node der neuen Liste dem letzten Node der ursprünglichen Liste (und umgekehrt).

Vervollständigen Sie die Methode `reverse()` in der Klasse `LinkedList`. Die Methode soll, wie oben definiert, die Liste umkehren. Achten Sie darauf, dass Sie wirklich die Reihenfolge der Nodes selbst umkehren. Es reicht nicht aus, die Reihenfolge der enthaltenen `int`-Werte umzukehren. Es müssen auch in der umgekehrten Liste dieselben Instanzen von `IntNodes` wie in der ursprünglichen Liste verwendet werden. Erstellen Sie also *keine* neuen `IntNodes` mit `new IntNode()`. In der Datei `“UmkehrungTest.java”` finden Sie einen einfachen Test.

Aufgabe 3: “KI” für das Ratespiel

In Übung 5 implementierten Sie ein Spiel, in welchem der Computer ein Wort auswählt und der Spieler dieses erraten muss. Dort war der Spieler der Benutzer des Programms. In dieser Aufgabe sollen Sie verschiedene “künstliche” Spieler entwickeln. Das heisst, anstelle des Menschen, der über die Konsole Tipps eingibt, werden die Tipps von (mehr oder weniger “intelligenten”) Programmen abgegeben. Ihr Ziel ist es, einen künstlichen Spieler zu entwickeln, der über mehrere Spiele hinweg die Wörter in so wenig Versuchen wie möglich errät.

Die Übungsvorlage enthält bereits den Code für das Ratespiel. Gegenüber Übung 5 ist dieser nun in verschiedene Klassen aufgeteilt. Die drei Hauptklassen sind `RateSpiel`, `Computer` und `Spieler`. Die Klasse `RateSpielApp` enthält eine `main`-Methode, welche das Spiel aufsetzt und durchführt. Durch die Aufteilung ist es möglich, mittels Vererbung Spieler mit unterschiedlichem Verhalten zu schreiben. Die Klasse `Spieler` enthält nämlich nur die Deklarationen der benötigten Methoden, aber keine (sinnvolle) Funktionalität. Subklassen von `Spieler` überschreiben diese Methoden und definieren damit das Verhalten eines Spielers.

Ein konkreter Spieler ist ebenfalls schon in der Vorlage vorhanden: der `KonsolenSpieler`. Dieser besitzt allerdings keine eigene “Intelligenz”, sondern holt sich die Tipps über die Konsole vom Benutzer. Ein `RateSpiel` mit einem `KonsolenSpieler` verhält sich also so wie das Spiel in Übung 5. Starten Sie die `RateSpielApp` und überzeugen Sie sich selbst¹.

Aufgabe 4: Klassenrätsel

In dieser Aufgabe sollen Sie zeigen, dass Sie mit Klassen und Vererbung umgehen können. Im Anhang [A](#) finden Sie ein Programm, welches Instanzen von Klassen erstellt und Methoden aufruft. Das Programm macht nichts Sinnvolles und dient nur dem Testen Ihrer Fähigkeiten. In Anhang [B](#) befinden sich die verwendeten Klassen, jedoch sind die Klassen noch nicht vollständig. Bei manchen der Klassen fehlt noch die `extends`-Klausel, welche angibt, dass eine Klasse von einer anderen Klasse erbt. Ihre Aufgabe ist es, die nötigen `extends`-Klauseln hinzuzufügen, so dass alles kompiliert und so dass die Ausgabe des Programms von Anhang [A](#) am Ende so aussieht wie im Anhang [C](#) gezeigt.

Der Code von Anhang [A](#) and Anhang [B](#) befindet sich in Ihrem `src`-Ordner. Zusätzlich enthält "KlassenTest.java" einen Unit-Test, welcher prüft, ob die Ausgabe des Programms dem Output aus Anhang [C](#) entspricht. Beachten Sie, dass Sie für diese Aufgabe **ausschliesslich** `extends`-Klauseln hinzufügen (diese kann es nur an den grauen Boxen aus Anhang [B](#) geben), kein anderer Code darf verändert werden.

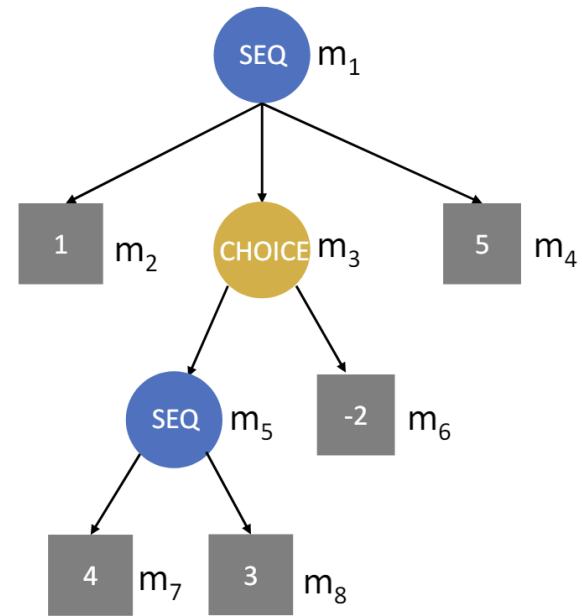
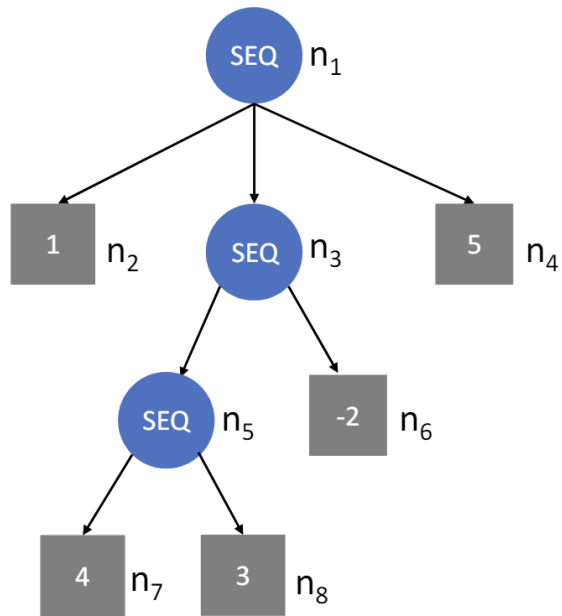
Tipp: Lösen Sie die Aufgabe zuerst auf Papier, ohne die Hilfe von Eclipse. Sobald Sie herausgefunden haben, welche Klassen von welchen Klassen erben, testen Sie Ihre Lösung in Eclipse. Dies hilft Ihnen, Ihr Wissen über Vererbung zu testen. In der Vergangenheit wurden ähnliche Aufgaben im schriftlichen Teil der Prüfung gestellt.

Aufgabe 5: Bonus

Tipp 1: Lesen Sie jeweils die Aufgabenstellung inklusive aller Teilaufgaben sorgfältig durch, bevor sie mit der Implementation der ersten Teilaufgabe beginnen.

Tipp 2: Es kann nützlich sein eine separate Klasse für die Entries zu erstellen, welche alle Eigenschaften eines Elements in der Warteschlange als Attribute enthält.

Probleme Lösen



In dieser Aufgabe verwenden wir **gerichtete azyklische Graphen**, um Programme zu repräsentieren. Der **Programmzustand** ist dabei immer durch ein **Tupel $(sum, counter)$** gegeben, wobei *sum* und *counter* ganze Zahlen sind. Programmzustände werden durch **ProgramState-Objekte** modelliert, wobei `ProgramState.getSum()` (bzw. `ProgramState.getCounter()`) dem ersten Element (bzw. dem zweiten Element) des Tupels entspricht.

Eine Ausführung des Programms manipuliert den Programmzustand. Das **Resultat** eines Programms ist gegeben durch den **erreichten Programmzustand**, nachdem **alle Operationen** im Programm **ausgeführt** wurden. Programme können **nichtdeterministisch** sein: Das heisst, für ein **einzelnes Programm** kann es für den **gleichen Startzustand** mehrere Programmausführungen geben, welche zu **unterschiedlichen Resultaten** führen.

Knoten in Graphen werden durch **Node-Objekte** modelliert. `Node.getSubnodes()` gibt die **Kinderknoten** als ein **Array** zurück (*m* ist genau dann ein Kinderknoten von *n*, wenn es eine ausgehende gerichtete Kante von *n* zu *m* gibt). Wir unterscheiden **drei Arten** von **Knoten**, wobei die Methode `Node.getType()` die Knotenart als String zurückgibt. Um ein Programm, welches durch den Knoten *n* repräsentiert wird, auszuführen, muss man den "Knoten *n* ausführen". Wir beschreiben nun die drei Knotenarten und jeweils die Ausführung der Knoten:

1. **Additionsknoten** (`Node.getType()` ist "ADD"): Solche Knoten besitzen einen **Additionswert** a gegeben durch `Node.getValue()` (eine **ganze Zahl**) und bei der Ausführung dieses Knotens wird der Programzustand von $(sum, counter)$ zu $(sum + a, counter + 1)$ aktualisiert. Die **Kinderknoten** von solchen Knoten werden bei der Ausführung **ignoriert**.
2. **Sequenzknoten** (`Node.getType()` ist "SEQ"): Bei der Ausführung eines Sequenzknoten n werden die **Kinderknoten** von n **nacheinander ausgeführt**. Die **Reihenfolge** in welcher die Kinderknoten ausgeführt werden **spielt keine Rolle**, da der erreichte Programzustand für jede Reihenfolge gleich ist. **`Node.getValue()` ist irrelevant.**
3. **Auswahlknoten** (`Node.getType()` ist "CHOICE"): Bei der Ausführung eines Auswahlknoten n wird ein **beliebiger Kinderknoten** von n **ausgewählt und ausgeführt**. **`Node.getValue()` ist irrelevant.** Diese Knoten führen zu **Nichtdeterminismus**.

Sie dürfen davon ausgehen, dass **Sequenz- und Auswahlknoten** immer **mindestens** einen **Kinderknoten** haben, und dass es **zwischen zwei Knoten immer höchstens einen Pfad** gibt. Die folgende Abbildung zeigt zwei Beispielgraphen, wobei Knoten mit der Beschriftung "SEQ" (bzw. "CHOICE") Sequenzknoten (bzw. Auswahlknoten) entsprechen und die Zahlen in Additionsknoten den Additionswerten entsprechen.

Implementieren Sie `GraphExecution.allResults(Node n, ProgramState initState)`, welche für den Startzustand `initState` alle möglichen Resultate für das Programm repräsentiert durch `n` zurückgibt. Die Resultate sollten als eine Liste von `ProgramState`-Objekten zurückgegeben werden (repräsentiert durch die Klasse `LinkedProgramStateList`). Die Reihenfolge der zurückgegebenen Liste spielt keine Rolle. Wenn das gleiche Resultat durch genau k verschiedene Ausführungen generiert werden kann, dann muss das Resultat k Mal in der zurückgegebenen Liste vorkommen. Zwei Ausführungen sind unterschiedlich, wenn es mindestens einen Knoten gibt, der in einer aber nicht in der anderen Ausführung ausgeführt wird.

Wir stellen zwei Testdateien zur Verfügung. “`GraphExecutionTest.java`” enthält Tests, welche wir an einer Prüfung geben würden. “`GradingGraphExecutionTest.java`” enthält Tests, welche wir zum Korrigieren einer Prüfung verwenden würden. Testen Sie ihre Lösung zuerst ausgiebig mit “`GraphExecutionTest.java`” (am besten fügen Sie selber neue Tests hinzu) und dann können Sie “`GradingGraphExecutionTest.java`” verwenden, um zu sehen, wie ihre Lösung an einer Prüfung abgeschnitten hätte.

- **ADD Node:** Enthalten value `a` und wir setzen `(sum, counter)` auf `(sum + a, counter + 1)`.
Kinderknoten werden bei der Ausführung ignoriert.

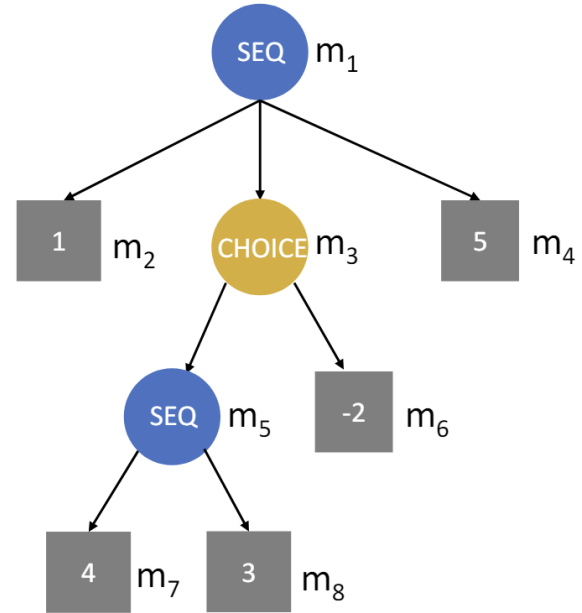
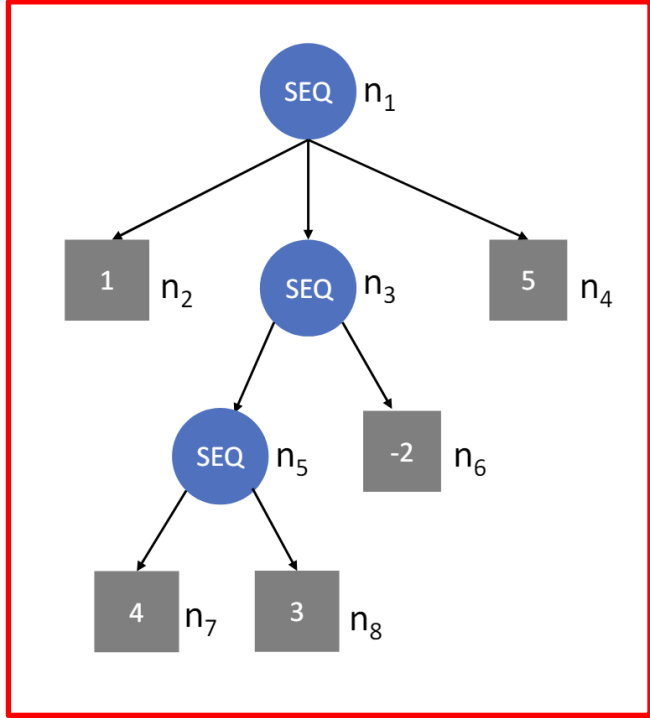


- **SEQ Node:** Kinderknoten werden nacheinander ausgeführt. Reihenfolge ist egal.
Das `value` Attribut wird ignoriert.

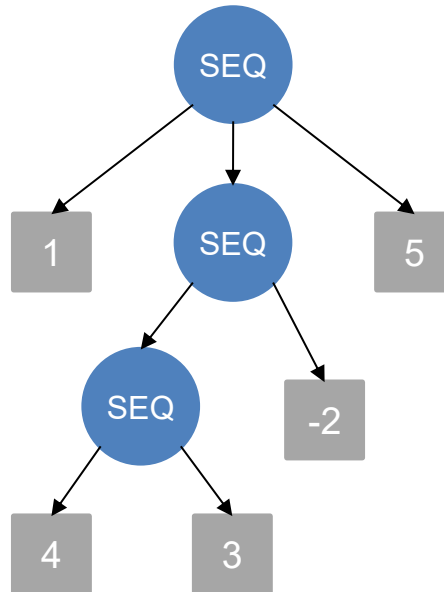


- **CHOICE Node:** Ein beliebiger Knoten wird ausgeführt. Das `value` Attribut wird ignoriert.

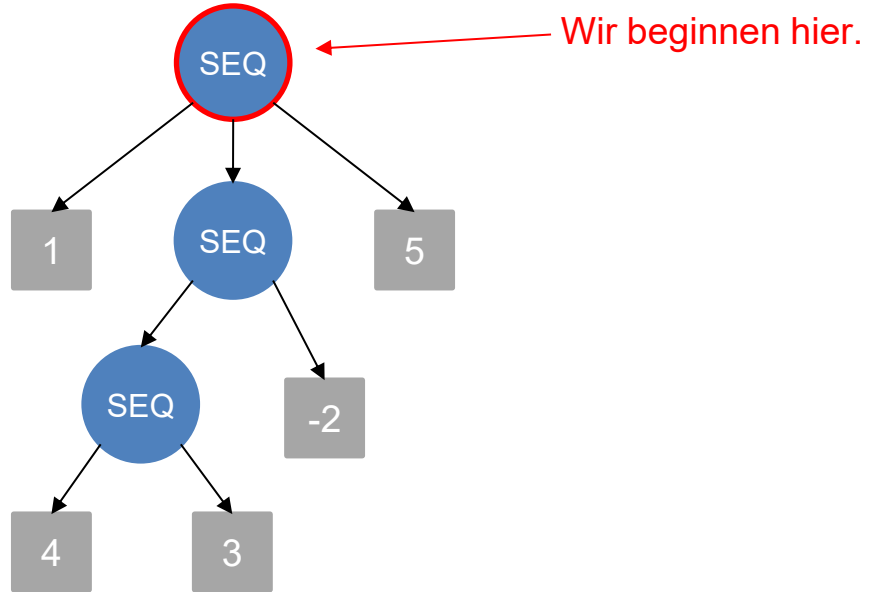


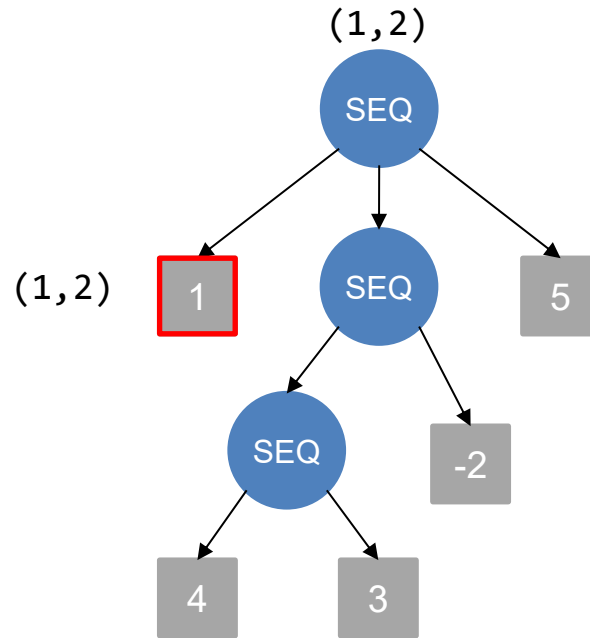


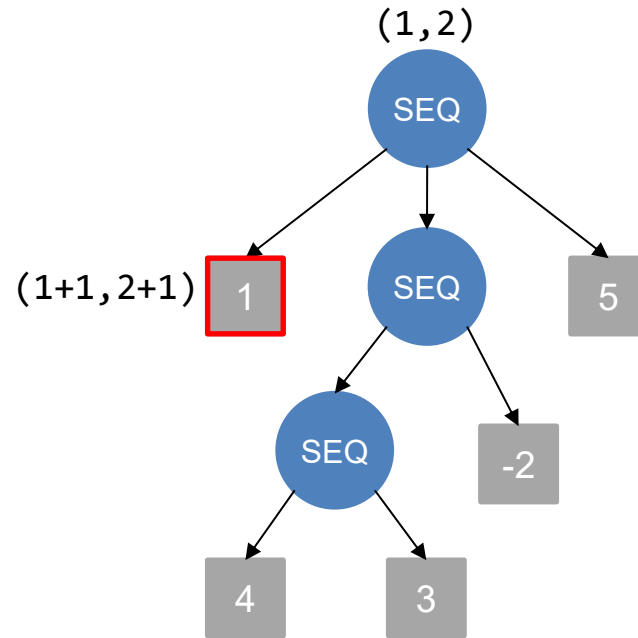
Startzustand: (1, 2)

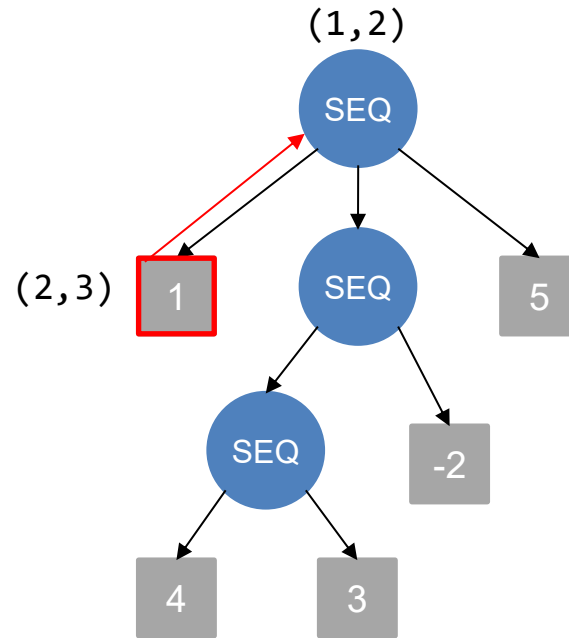


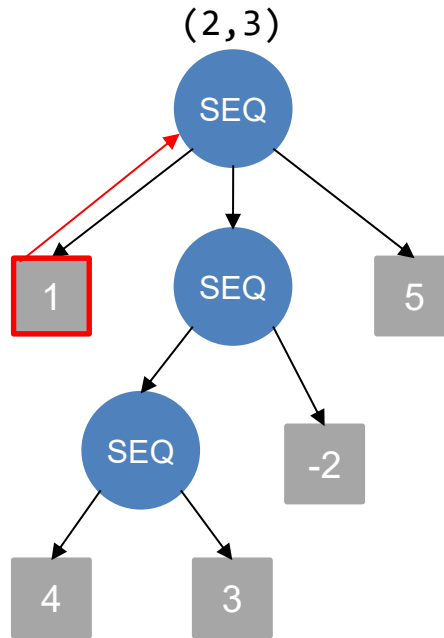
Startzustand: (1, 2)

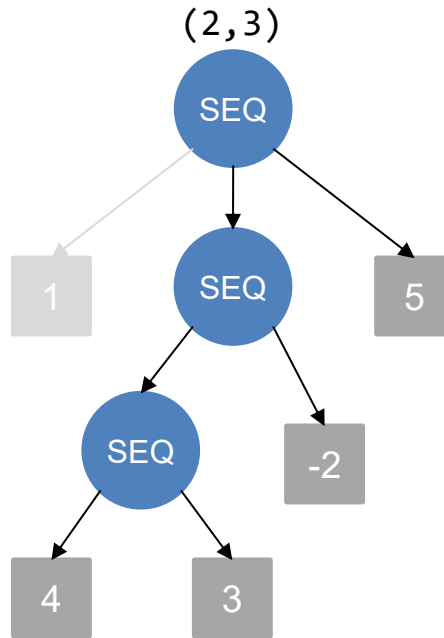


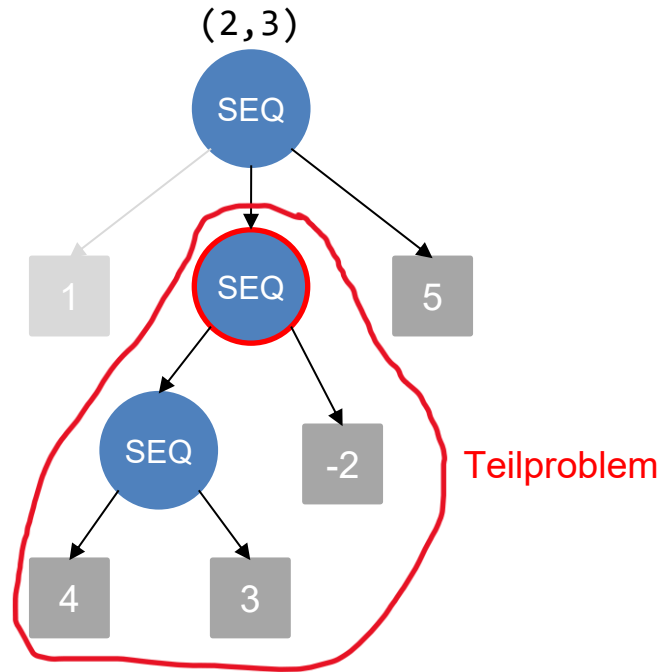


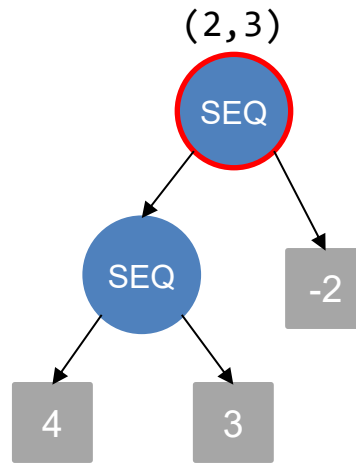
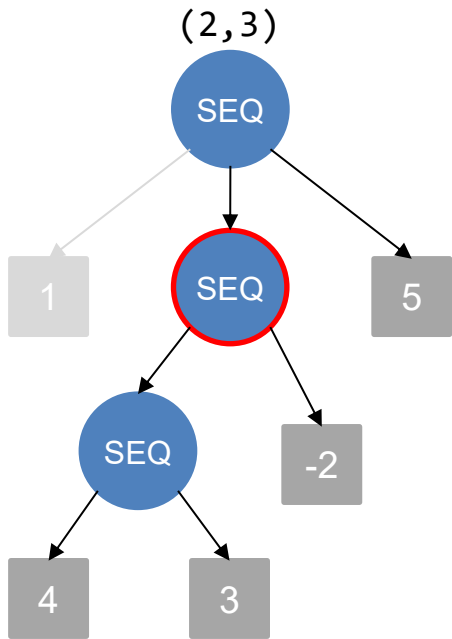


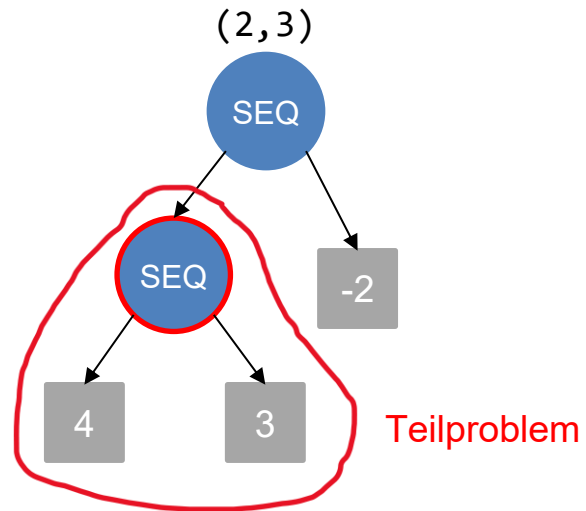
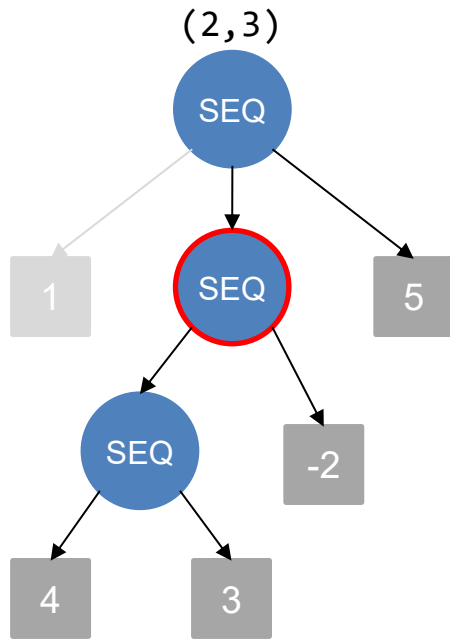


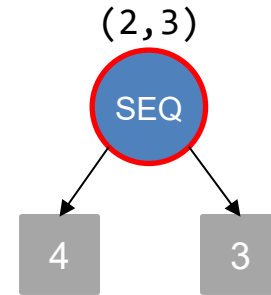
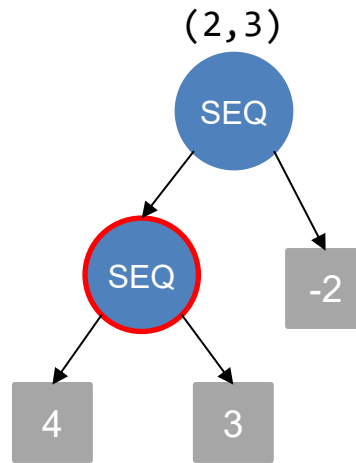
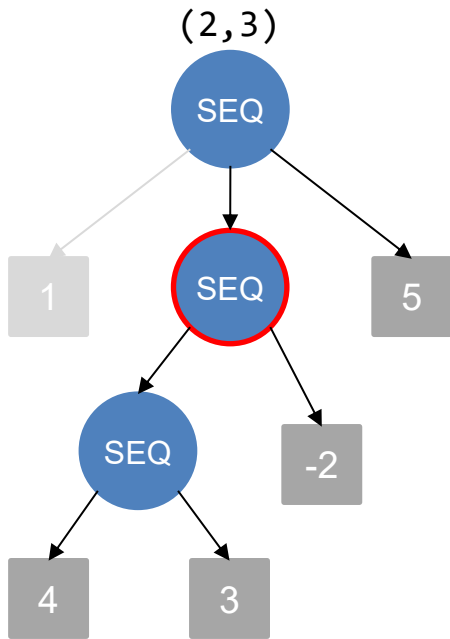


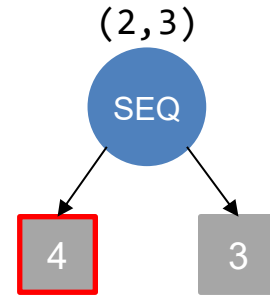
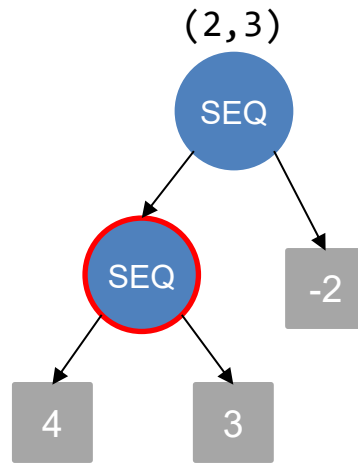
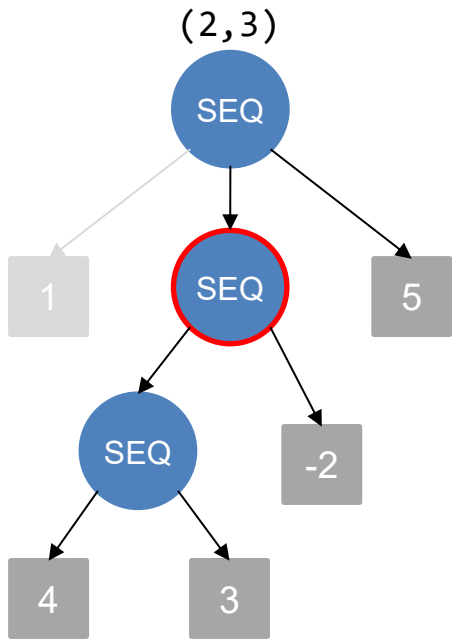


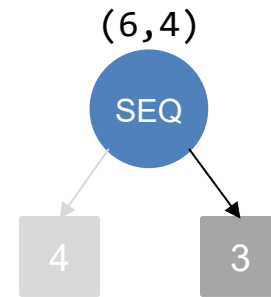
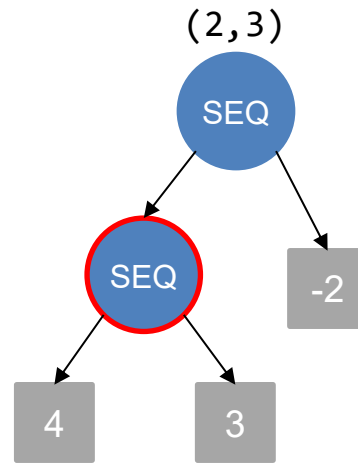
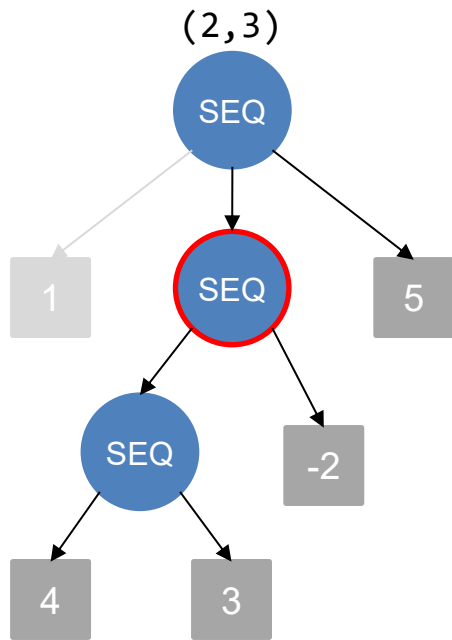


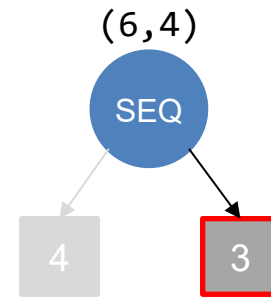
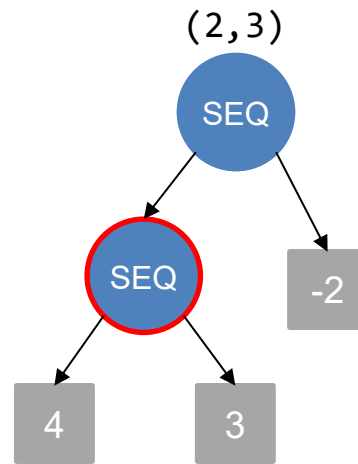
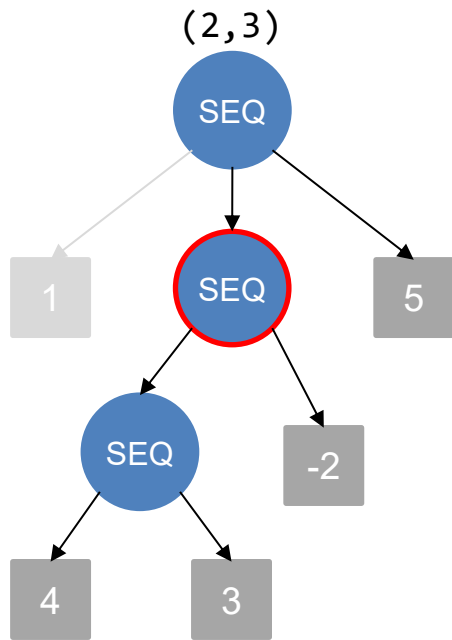


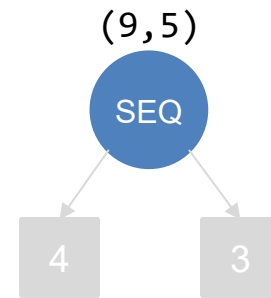
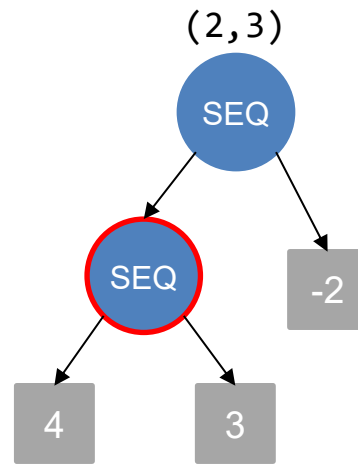
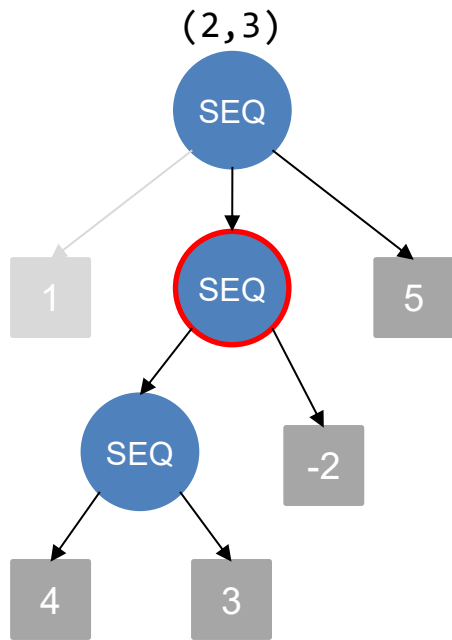


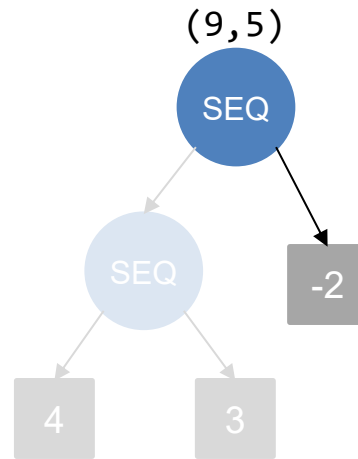
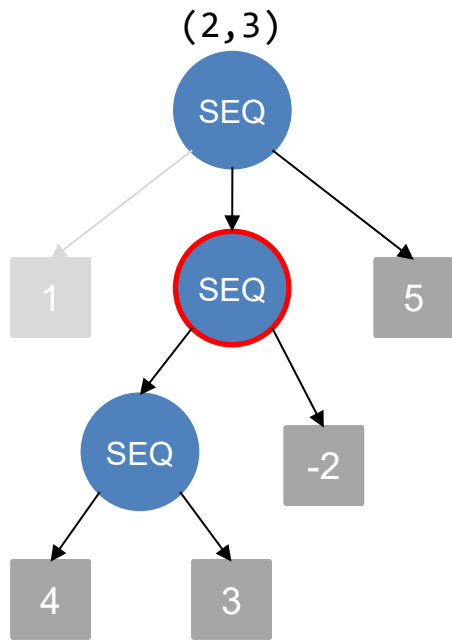


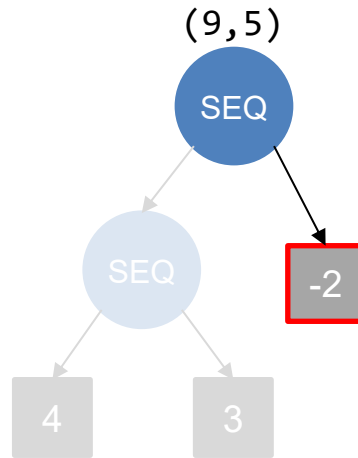
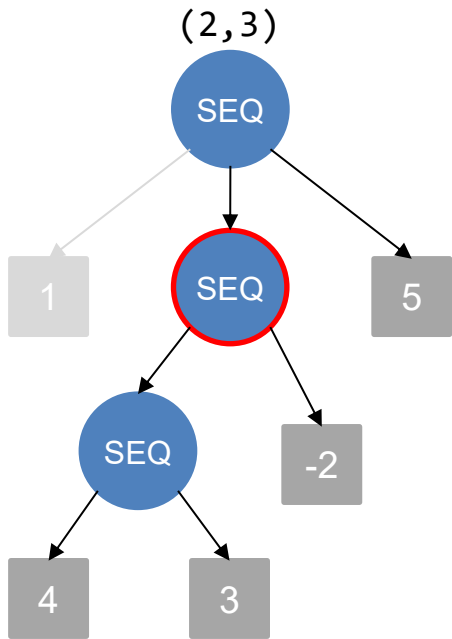


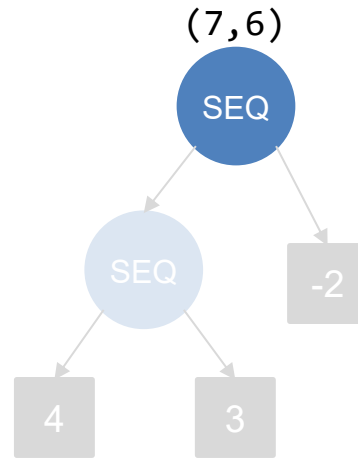
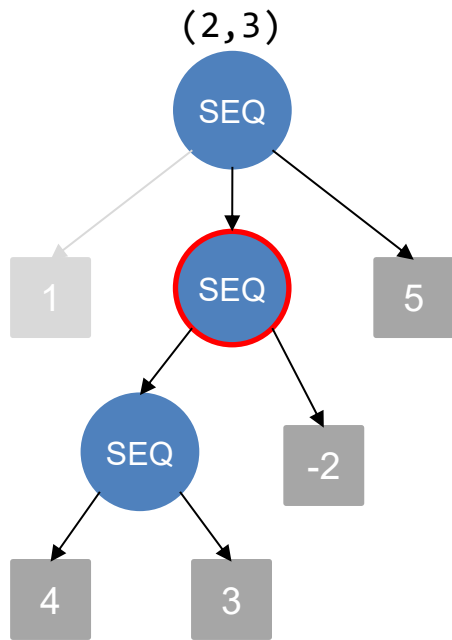


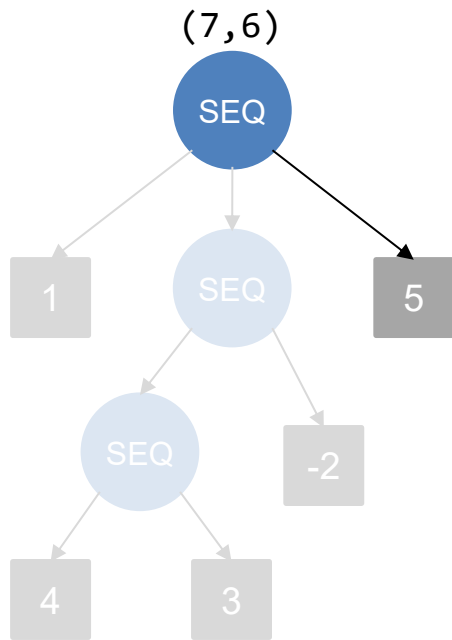


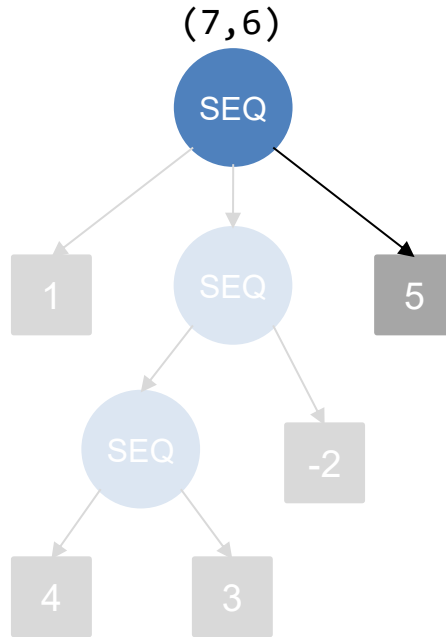


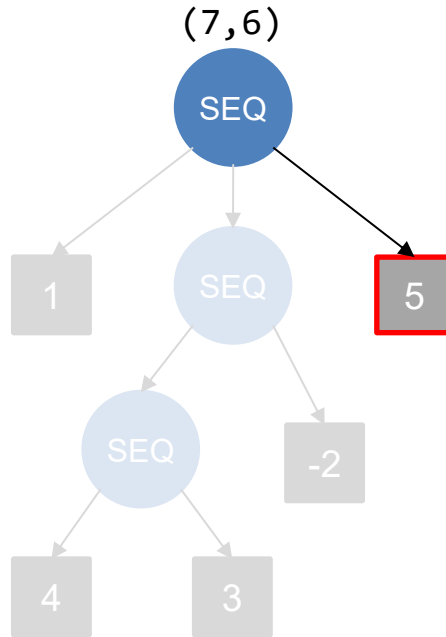


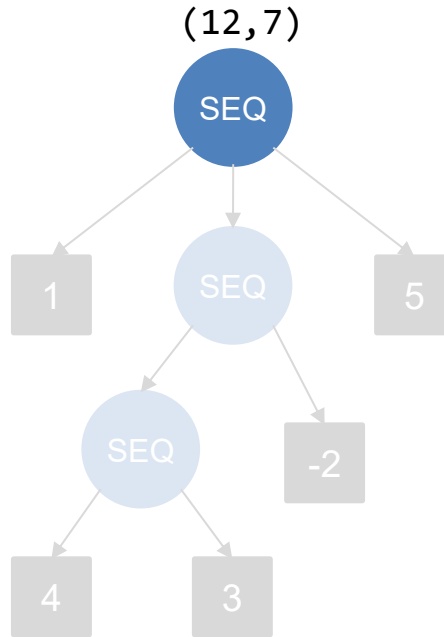


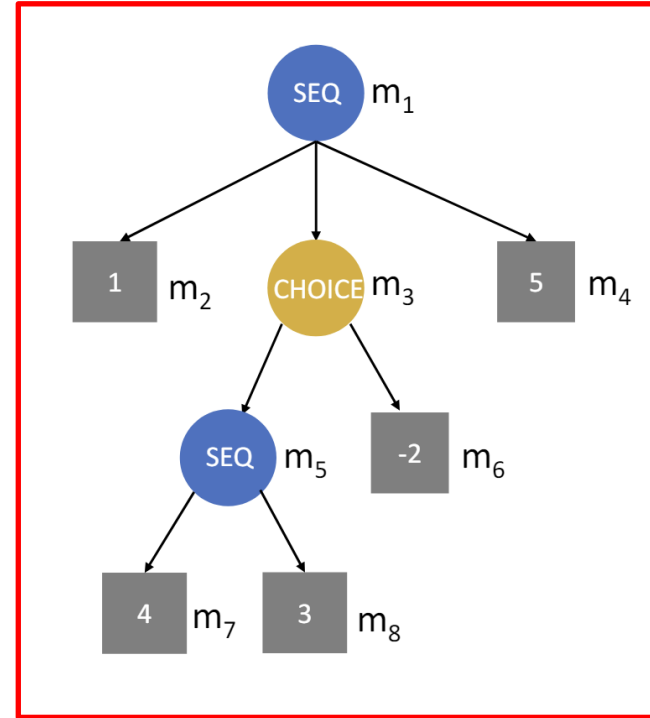
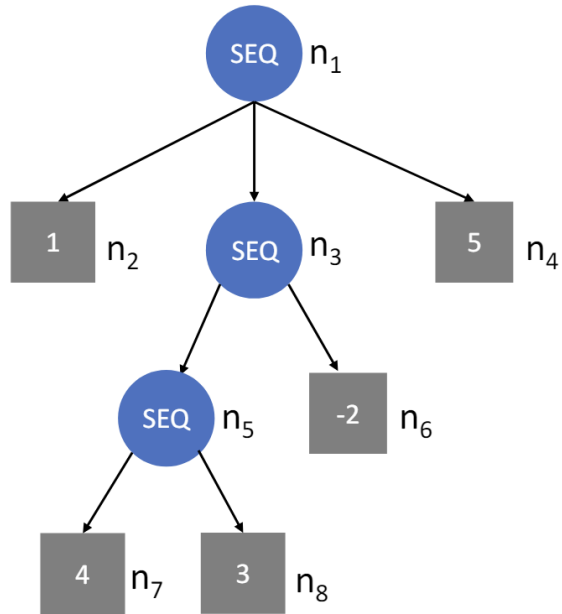




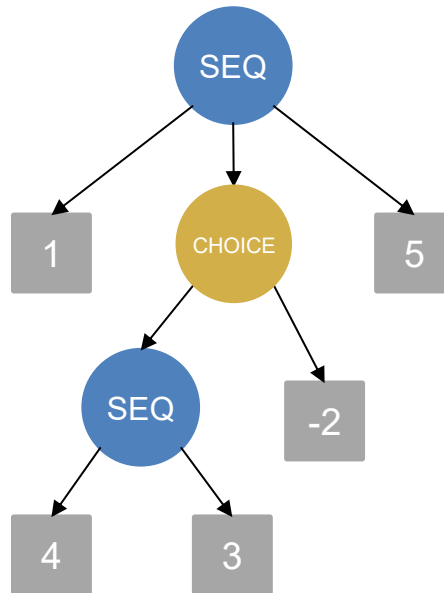




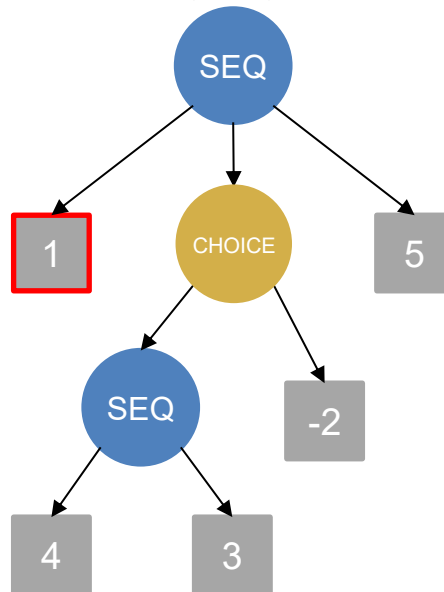




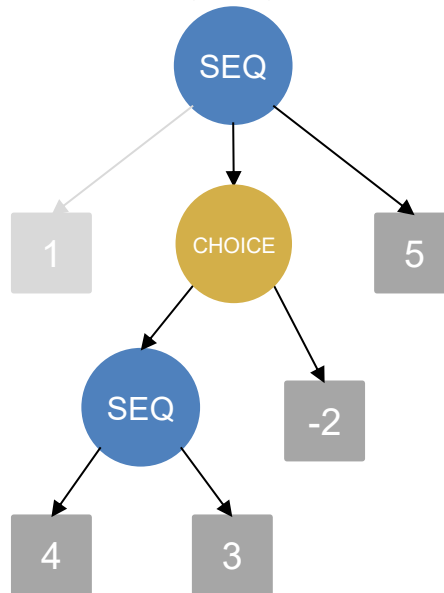
Startzustand: $(0,0)$



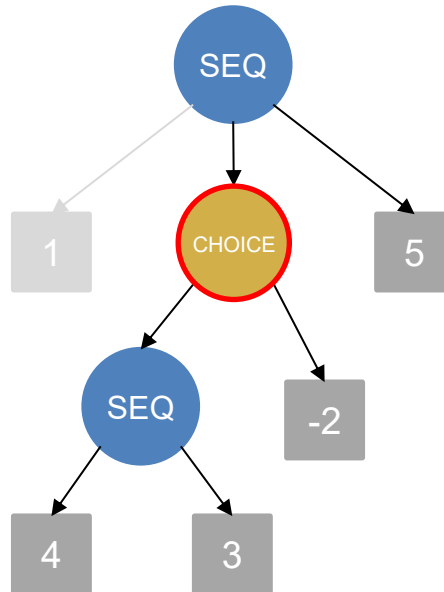
Startzustand: $(0,0)$



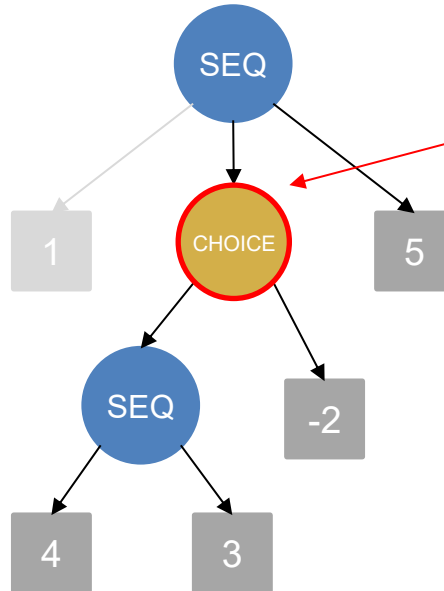
Startzustand: (1,1)



Startzustand: (1,1)



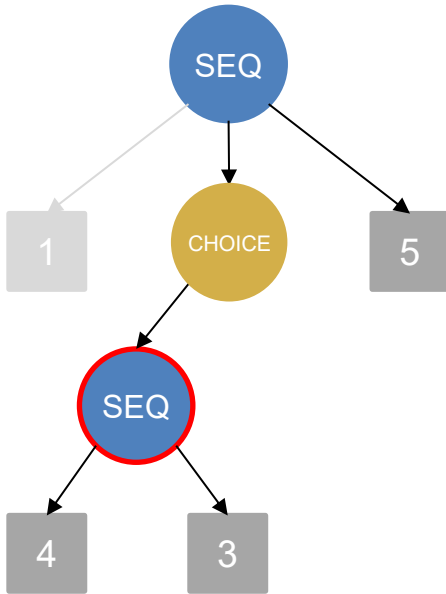
Startzustand: (1,1)



Wir haben jetzt die
Wahl zwischen
zwei Kinderknoten.

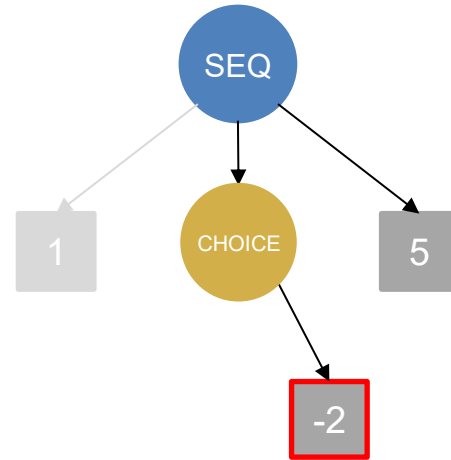
Resultat 1

(1,1)



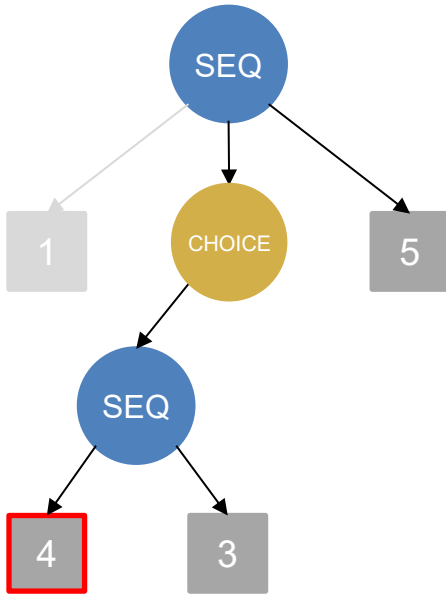
Resultat 2

(1,1)



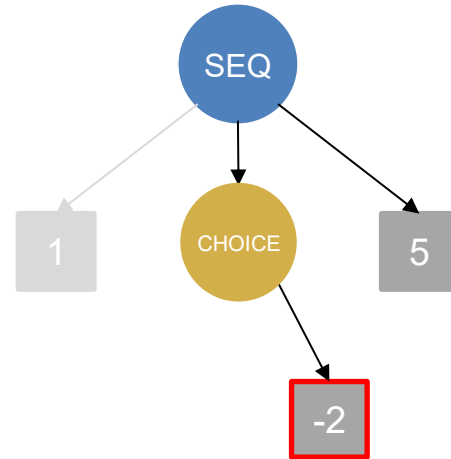
Resultat 1

(1,1)



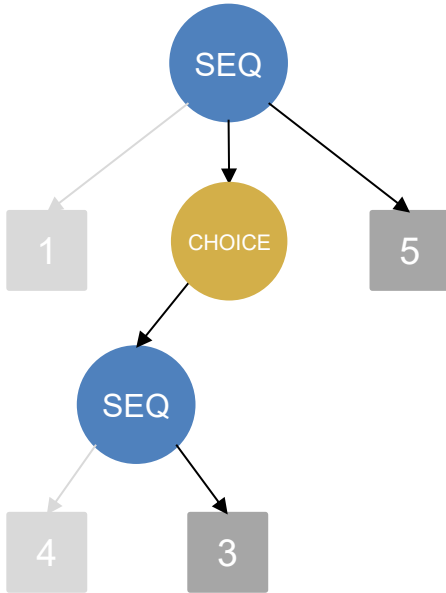
Resultat 2

(1,1)



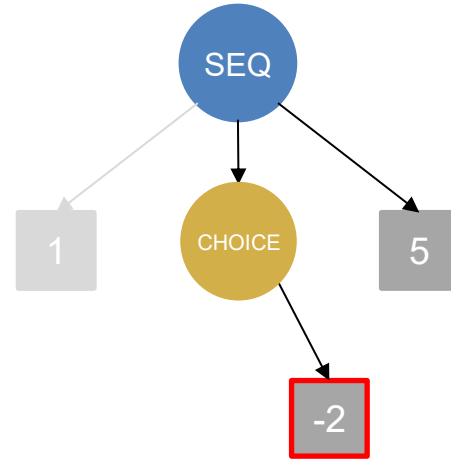
Resultat 1

(5, 2)



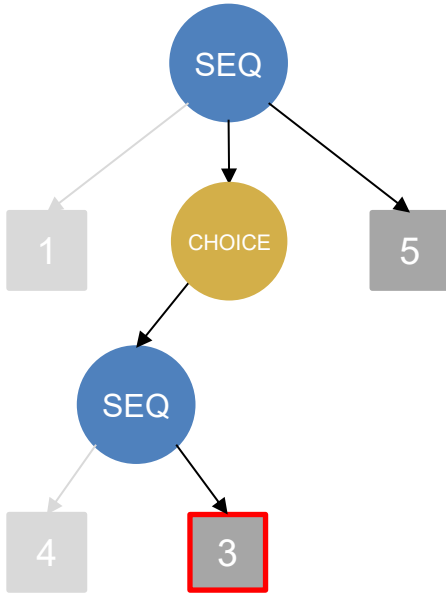
Resultat 2

(1, 1)



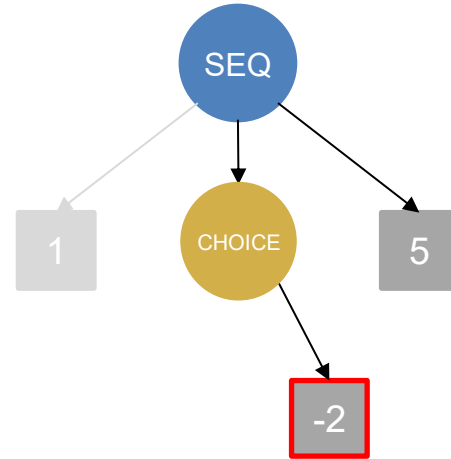
Resultat 1

(5, 2)



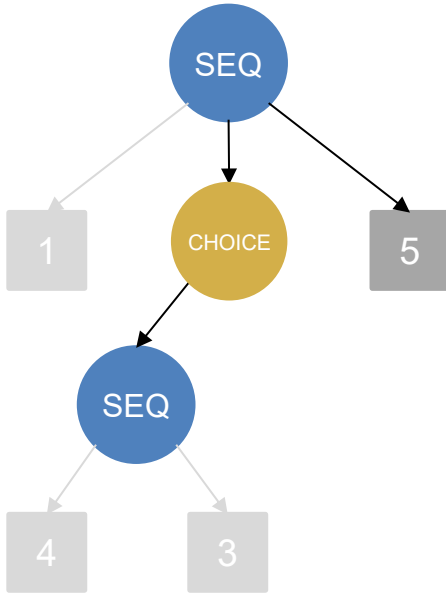
Resultat 2

(1, 1)



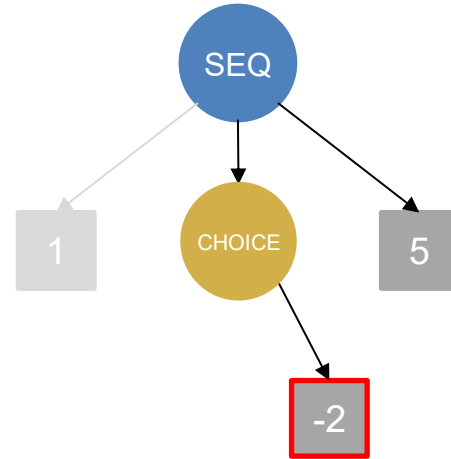
Resultat 1

(8, 3)



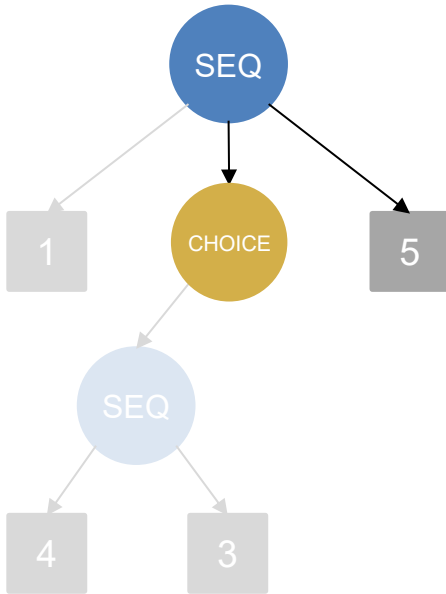
Resultat 2

(1, 1)



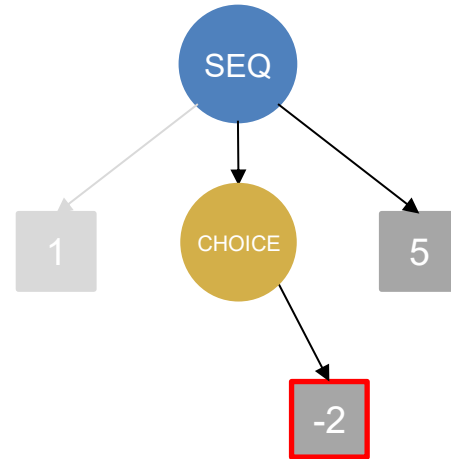
Resultat 1

(8, 3)



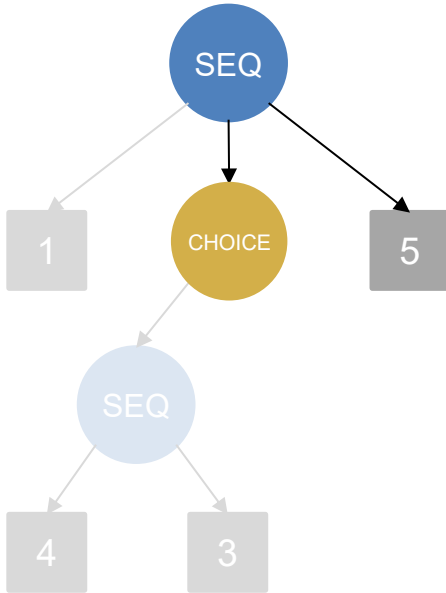
Resultat 2

(1, 1)



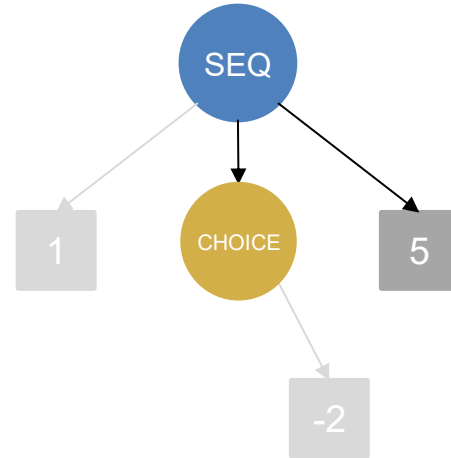
Resultat 1

(8, 3)



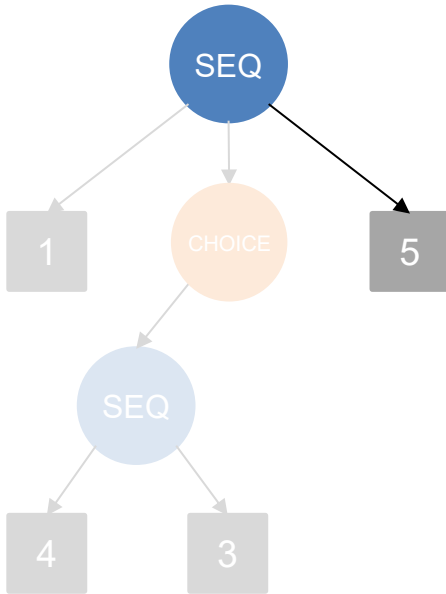
Resultat 2

(-1, 2)



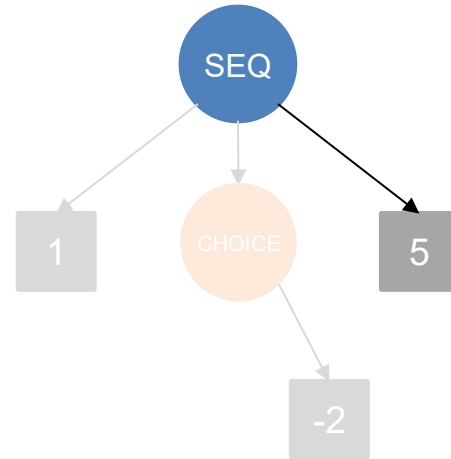
Resultat 1

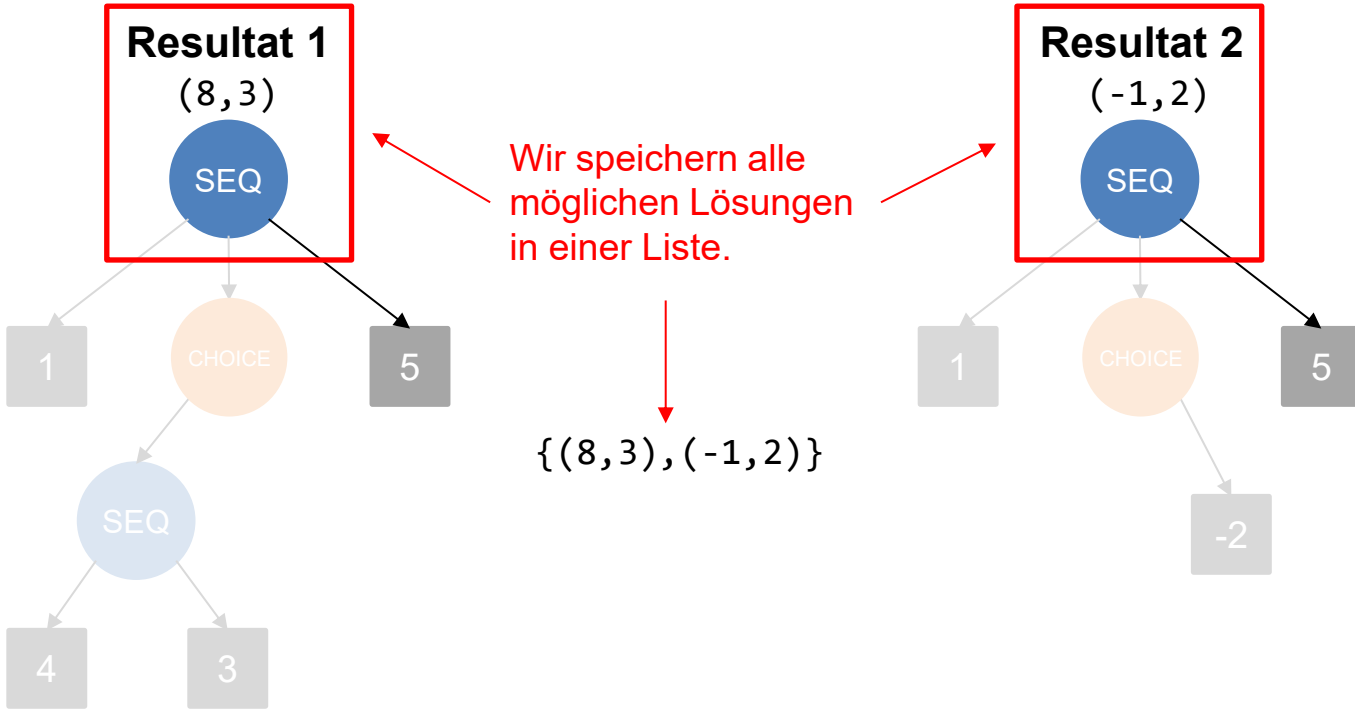
(8, 3)



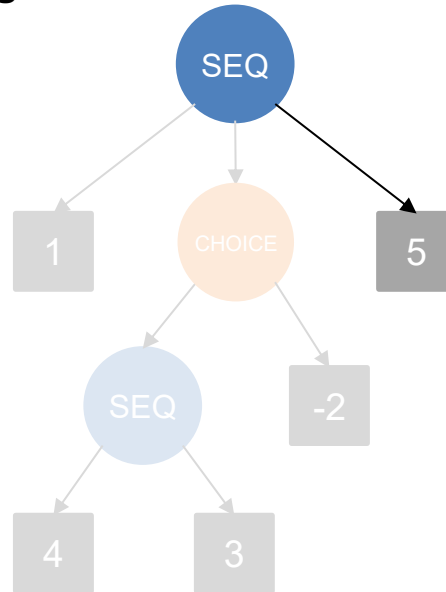
Resultat 2

(-1, 2)



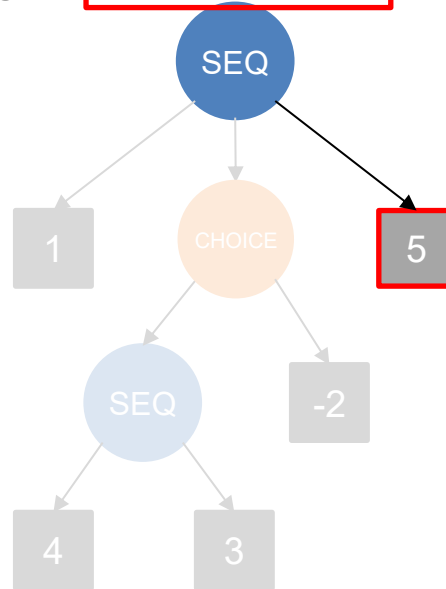


Lösungen: $\{(8, 3), (-1, 2)\}$

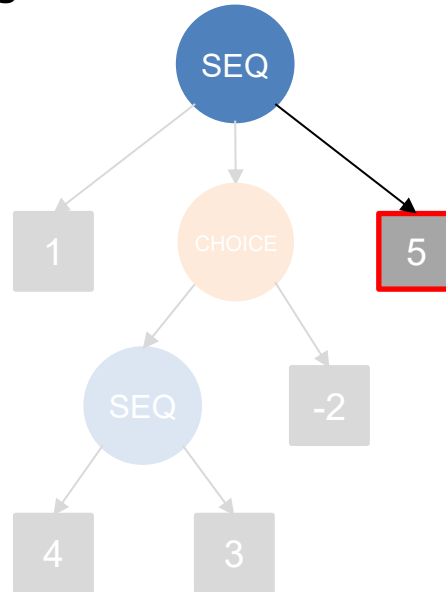


Lösungen: $\{(8, 3), (-1, 2)\}$

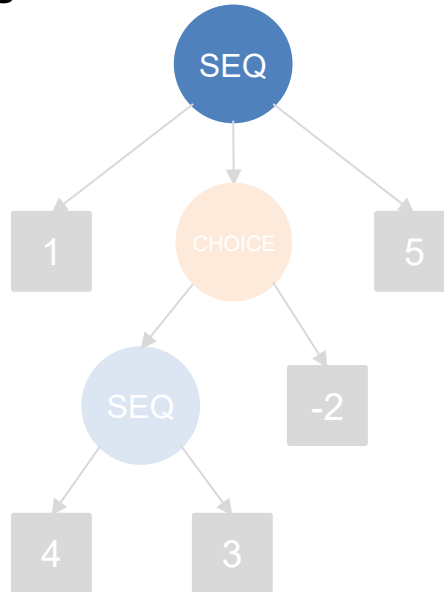
Wir müssen jedes
Zwischenresultat in
der Liste updaten.



Lösungen: $\{(8+5, 3+1), (-1+5, 2+1)\}$



Lösungen: $\{(13,4), (4,3)\}$



Wie lösen wir das Problem?

- Wir nutzen eine **Helfermethode** `allResultsGo` welche statt nur einem Programmstate eine Liste von Programmstates als Parameter hat.
- **ADD:** Für alle Zwischenresultate addiere `value` dazu, erhöhe den Counter um 1 und füge das Resultat zu `next` hinzu.
- **SEQ:** Rufe für alle Kinderknoten die Methode `allResultsGo` auf. Wir speichern die zurückgegebene Liste und nutzen diese als Parameter für den nächsten Kinderknoten.
- **CHOICE:** Rufe für alle Kinderknoten die Methode `allResultsGo` auf. Wir berechnen die Resultate für jeden Kinderknoten separat und fügen die Listen zusammen.

```
public static LinkedProgramStateList allResultsGo(Node n, LinkedProgramStateList states) {  
    if (n.getType().equals("ADD")) {  
        LinkedProgramStateList next = new LinkedProgramStateList();  
        for (int i = 0; i < states.size; i += 1) {  
            ProgramState state = states.get(i);  
            next.addLast(new ProgramState(state.getSum() + n.getValue(), state.getCounter() + 1));  
        }  
        return next;  
    }  
    (...)  
}
```

Neue Liste wird
erstellt, da wir nicht
states ändern wollen.

```
public static LinkedProgramStateList allResultsGo(Node n, LinkedProgramStateList states) {  
    if (n.getType().equals("ADD")) {  
        LinkedProgramStateList next = new LinkedProgramStateList();  
        for (int i = 0; i < states.size; i += 1) {  
            ProgramState state = states.get(i);  
            next.addLast(new ProgramState(state.getSum() + n.getValue(), state.getCounter() + 1));  
        }  
        return next;  
    }  
    (...)  
}
```

Für jedes Zwischenresult in der Liste states wird value zur Summe hinzugefügt und der Counter erhöht.

```
public static LinkedProgramStateList allResultsGo(Node n, LinkedProgramStateList states) {  
    (...)  
    } else if (n.getType().equals("SEQ")) {  
        LinkedProgramStateList next = states;  
        for (Node ch : n.getSubnodes()) { //Recursively update the results  
            next = allResultsGo(ch, next);  
        }  
        return next;  
    }  
    (...)  
}
```

Wir verändern die Liste auf welche states verweist **nicht**, weil wir bei ADD nodes eine neue Liste erstellen. Hier wird aber **keine** Kopie erstellt!



```
public static LinkedProgramStateList allResultsGo(Node n, LinkedProgramStateList states) {  
    (...)  
    } else if (n.getType().equals("SEQ")) {  
        LinkedProgramStateList next = states;  
        for (Node ch : n.getSubnodes()) {  
            next = allResultsGo(ch, next);  
        }  
        return next;  
    }  
    (...)  
}
```

Wir rufen für jeden
Kinderknoten die
Methode rekursiv auf.

```
public static LinkedProgramStateList allResultsGo(Node n, LinkedProgramStateList states) {  
    (...)  
    } else if (n.getType().equals("CHOICE")) {  
        LinkedProgramStateList next = new LinkedProgramStateList();  
        for (Node ch : n.getSubnodes()) {  
            LinkedProgramStateList results = allResultsGo(ch, states);  
            for (int i = 0; i < results.size; i += 1) {  
                next.addLast(results.get(i));  
            }  
        }  
        return next;  
    }  
    return null;  
}
```

Für jeden Kinderknoten wird eine Liste zurückgegeben und wir fügen diese dann zusammen.